QUICKBITE

ONLINE FOOD ORDERING SYSTEM

Microservices Development and Deployment

SEZG583 | Scalable Services | Assignment 1

Jyoti | 2025MT03068



QuickBite customer, restaurant and delivery journey | Concept illustration

Contents

Section	Coverage	Page
1 Work products and application access links	Highlighted repositories, rubric map, author details and submission package	3
2 Architecture and business capabilities	Application design, executive summary, capabilities and illustrated business journey	7
Final implemented architecture	Source-verified topology and runtime evidence	9
3 System operations and collaboration	Commands, queries and REST collaboration	10
3a Core Scenarios	Service collaboration, customer Web UI and Swagger	10
3b QuickBite API testing with Postman	Implementation and functional testing; original API responses	12
4 Implementation and project structure	Implementation steps, service files, data ownership and project structure	24
5 Local execution and Docker deployment	Execution instructions, prerequisites, local verification and Docker evidence	26
6 Kubernetes deployment	Deployment plan, build/load/apply commands, Dashboard and recovery	28

Section	Coverage	Page
Runtime configuration	CPU, memory, HPA, PDB, ports, storage and probes	29
7 Scalability enhancement using HPA	Observed scaling and recovery; HPA baseline, scale-out and scale-in	33
8 Repositories	Single project repository, service folders and historical GitHub evidence	35
9 CI and CD execution	Tests, builds, runner and Kubernetes deployment job	39
References	Technical documentation, project sources and assignment brief	42

1 Work products and application access links

QuickBites is the single project repository. All service-source links below open folders within it.

The root CI/CD workflow is the execution entry point; nested service workflow files are retained source copies.

Work Products and Access Links

Work Product	Access Link
Main Project Repository	https://github.com/erjyotisrvtv-lab/QuickBites
Gateway Service Source	https://github.com/erjyotisrvtv-lab/QuickBites/tree/main/services/gateway
Catalog Service Source	https://github.com/erjyotisrvtv-lab/QuickBites/tree/main/services/catalog
Orders Service Source	https://github.com/erjyotisrvtv-lab/QuickBites/tree/main/services/orders
Delivery Service Source	https://github.com/erjyotisrvtv-lab/QuickBites/tree/main/services/delivery
Evidence_assignment	QuickBites/Evidence_assigmnet.zip at main · erjyotisrvtv-lab/QuickBites
Project Zip folder	Release Jyoti_2025MT03068_Quickbite.zip · erjyotisrvtv-lab/QuickBites
Demonstration Video	Release QuickBite Assignment 1 Final Submission · erjyotisrvtv-lab/QuickBites
Project README	https://github.com/erjyotisrvtv-lab/QuickBites/blob/main/README.md

Work Product	Access Link
Postman Collection	https://github.com/erjyotisrvtv-lab/QuickBites/blob/main/postman/QuickBite.postman_collection.json
Kubernetes Manifests	https://github.com/erjyotisrvtv-lab/QuickBites/tree/main/deploy/kubernetes
Catalog Dockerfile	https://github.com/erjyotisrvtv-lab/QuickBites/blob/main/services/catalog/Dockerfile
Orders Dockerfile	https://github.com/erjyotisrvtv-lab/QuickBites/blob/main/services/orders/Dockerfile
Delivery Dockerfile	https://github.com/erjyotisrvtv-lab/QuickBites/blob/main/services/delivery/Dockerfile
Gateway Dockerfile	https://github.com/erjyotisrvtv-lab/QuickBites/blob/main/services/gateway/Dockerfile
Integration Workflow	https://github.com/erjyotisrvtv-lab/QuickBites/tree/main/.github/workflows
Catalog Workflow Source Folder	https://github.com/erjyotisrvtv-lab/QuickBites/tree/main/services/catalog/.github/workflows
Orders Workflow Source Folder	https://github.com/erjyotisrvtv-lab/QuickBites/tree/main/services/orders/.github/workflows

Work Product	Access Link
Delivery Workflow Source Folder	https://github.com/erjyotisrvtv-lab/QuickBites/tree/main/services/delivery/.github/workflows
Gateway Workflow Source Folder	https://github.com/erjyotisrvtv-lab/QuickBites/tree/main/services/gateway/.github/workflows
GitHub Actions Dashboard	https://github.com/erjyotisrvtv-lab/QuickBites/actions
GitHub Actions Deployment Run	https://github.com/erjyotisrvtv-lab/QuickBites/actions/runs/42574715176
Integration CI/CD workflow file	https://github.com/erjyotisrvtv-lab/QuickBites/blob/main/.github/workflows/quickbite-ci.yml
Catalog CI Source Copy	https://github.com/erjyotisrvtv-lab/QuickBites/blob/main/services/catalog/.github/workflows/ci.yml
Orders CI Source Copy	https://github.com/erjyotisrvtv-lab/QuickBites/blob/main/services/orders/.github/workflows/ci.yml
Delivery CI Source Copy	https://github.com/erjyotisrvtv-lab/QuickBites/blob/main/services/delivery/.github/workflows/ci.yml
Gateway CI Source Copy	https://github.com/erjyotisrvtv-lab/QuickBites/blob/main/services/gateway/.github/workflows/ci.yml

Assignment requirements and evidence

Requirement	Where addressed
Part 1 Design	Sections 2–3 with inline evidence. Application use case, scalable architecture, commands and queries, business capabilities and collaboration.
Part 2 Implementation	Sections 3–4 and 8 with inline evidence. FastAPI, REST, separate data stores, independent repositories, project structure and API snapshots.
Part 3 Deployment	Sections 5–7 with inline evidence. Separate Docker containers, local Minikube, deployment plan, Dashboard analysis and execution instructions.
Optional CI/CD	Section 9 with inline evidence. GitHub Actions, tests, builds, runner and deployment evidence.

Requirement	Where addressed
Submission deliverables	Cover and Section 1 with inline evidence. Member details, repository URLs, demo video and single LMS ZIP naming.

2 Architecture and business capabilities

Executive summary

QuickBite separates restaurant browsing, order management, and delivery management into FastAPI services behind an API gateway. Orders and Delivery each own a PostgreSQL database. Docker Compose runs the local stack, and Minikube runs the Kubernetes deployment in the food-delivery namespace.

Postman captures document restaurant and menu reads, order creation and retrieval, and delivery creation, retrieval and duplicate handling. Kubernetes evidence records four application Deployments with two ready replicas each at baseline, two database pods, bound storage, and an Orders autoscaling test. GitHub Actions evidence covers test, build and deployment jobs.

A customer selects a restaurant, reads its menu, places an order, and uses the resulting order ID to request delivery and retrieve order status. The browser calls the gateway over HTTP. The gateway forwards REST requests to the business services, while Orders and Delivery persist their own records.

Capability and owner	Commands	Queries and communication
Restaurant discovery · Catalog · 3001	None in this use case	List restaurants and retrieve a menu. Catalog supplies the restaurant and menu data.
Order management · Orders · 3002	Place order	Retrieve order by ID or list orders. Orders records items, totals and status in Orders PostgreSQL.
Delivery management · Delivery · 3003	Create delivery for an order ID	Retrieve delivery by ID. Delivery records its own status and rejects duplicate creation for an existing order.
Client access · Gateway · 8080	Forward write requests	Serve the UI and forward read requests to Catalog, Orders or Delivery.

Business capabilities and data ownership

The customer obtains restaurant and menu identifiers before posting an order. **Orders calls Catalog** to validate the menu items and calculate the total from catalog prices, then persists the order. **Delivery validates** the supplied order ID by calling Orders before creating its own record. Each service owns its PostgreSQL writes and returns its own status.

Figure 1 is the final implemented architecture, checked against the GitHub source and runtime evidence. Orders validates menu items through Catalog. **Delivery validates** the order through Orders. The gateway provides the client entry point.

QuickBite illustrated business journey

The three scenes connect the customer journey to the implemented service boundaries. The illustrations explain the business context; the following architecture diagrams and original screenshots document the technical implementation.



Concept illustration: browse a restaurant, place an order, then request delivery. The chef and rider are narrative characters, not additional implemented services.

BROWSE | CATALOG	PLACE | ORDERS	REQUEST | DELIVERY
The customer retrieves restaurants and the r2 menu through Gateway. Catalog provides menu identifiers and prices on port 3001.	Orders validates menu items with Catalog, calculates the total and persists the order in its own PostgreSQL database. Service port: 3002.	Delivery validates the order with Orders and stores a queued delivery in its own PostgreSQL database. Repeated creation for the same order returns 409. Service port: 3003.

Gateway is the common client entry point on port 8080. Orders and Delivery own separate data stores; the service collaboration remains HTTP REST.

Final implemented architecture

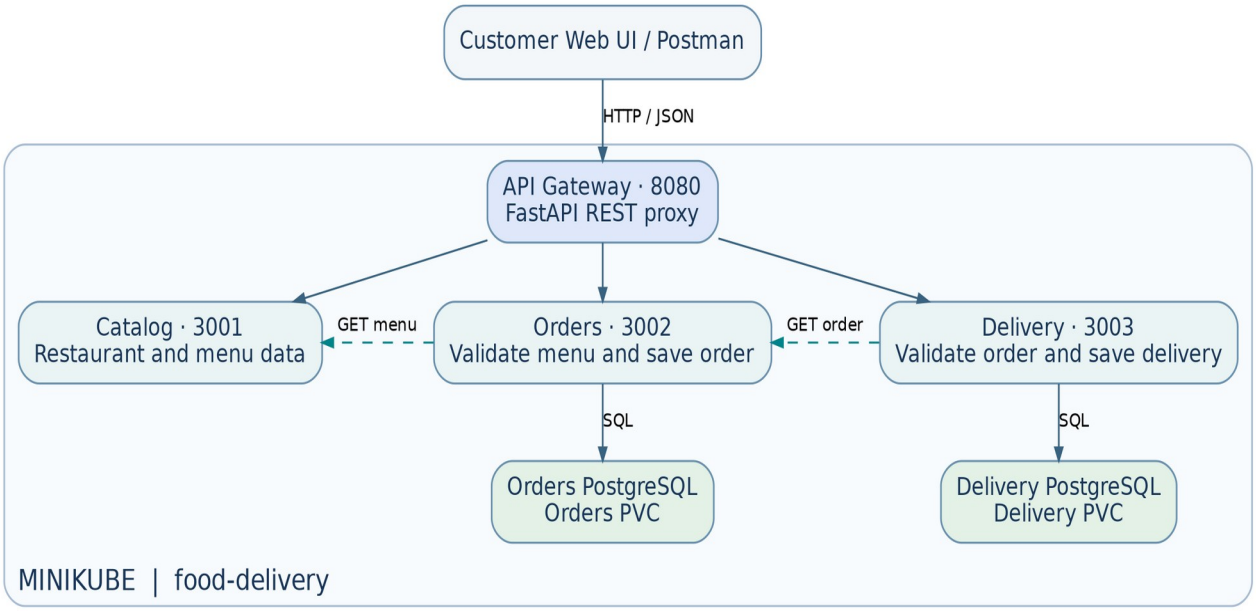


Figure 1. Final implemented architecture, checked against GitHub commit ce8c3f6831ecdff891996415cfafe86724c6e47a. Solid arrows show client routing and database access. Dashed arrows show Orders querying Catalog and Delivery validating an order with Orders.

Diagram component	What to inspect	Exhibits
Client and gateway	Web UI plus gateway-prefixed requests. Service output confirms gateway port 8080.	A10, A17–A23, A25

Diagram component	What to inspect	Exhibits
Catalog	Restaurant list and Corner Pizza menu on the Catalog route.	A10, A17–A18
Orders and its database	Creation and matching lookup, Orders service port, database pod and Bound claim.	A09–A10, A16, A19–A20
Delivery and its database	Queued delivery, lookup and duplicate conflict, Delivery service port and Bound claim.	A09–A10, A16, A21–A23
Minikube runtime	Four Deployments at baseline, Dashboard, HPA, PDB and resource metrics.	A09–A16

Diagram component	What to inspect	Exhibits
Build and release	Separate Docker images and Compose containers. Actions shows deployment into Minikube.	A30–A31, A43–A46

3 System operations and collaboration

3a Core Scenarios

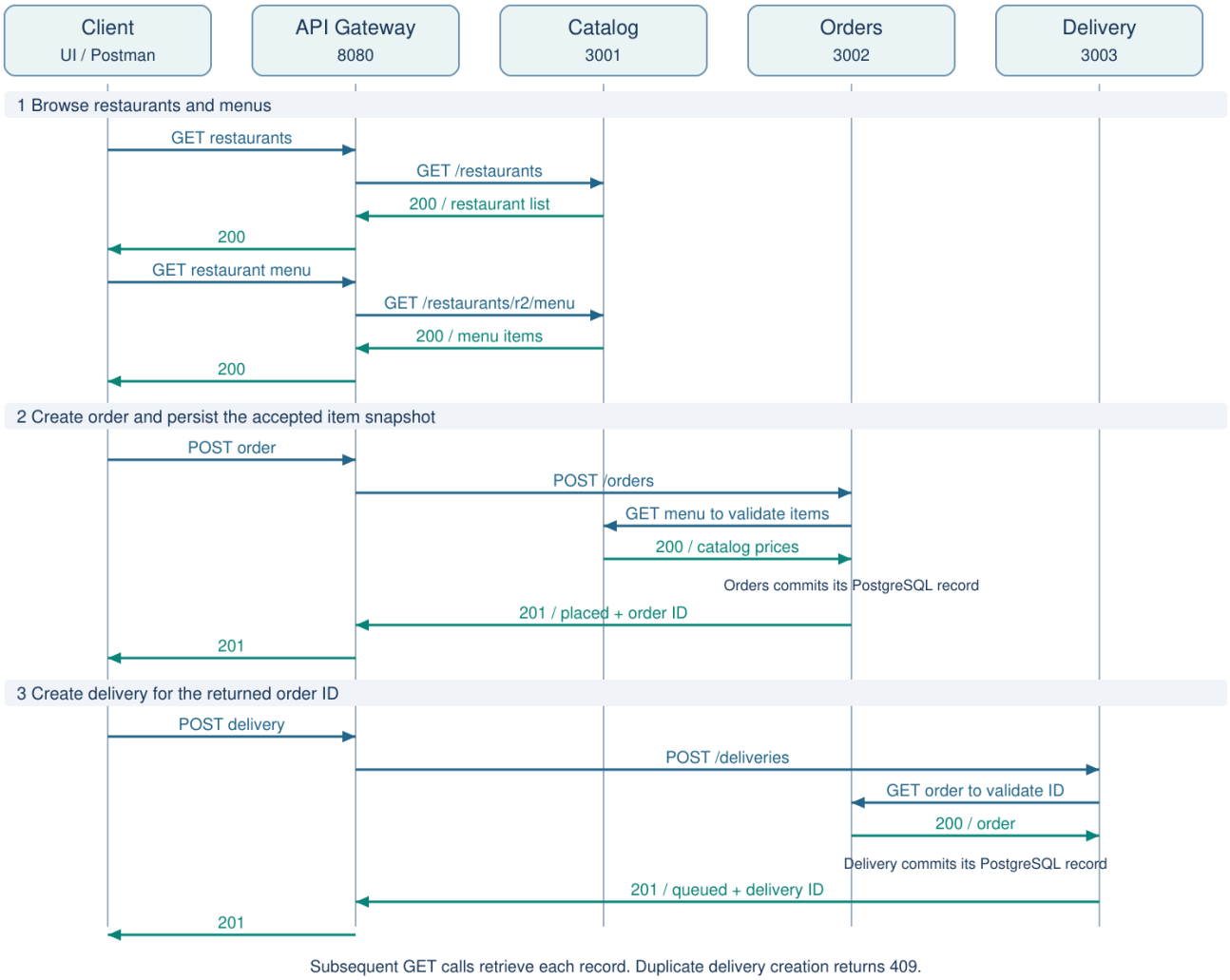


Figure 2. Implemented REST collaboration derived from the source. A17–A23 document the corresponding client requests and responses. Orders and Delivery validate their upstream inputs before committing records to their own PostgreSQL databases.

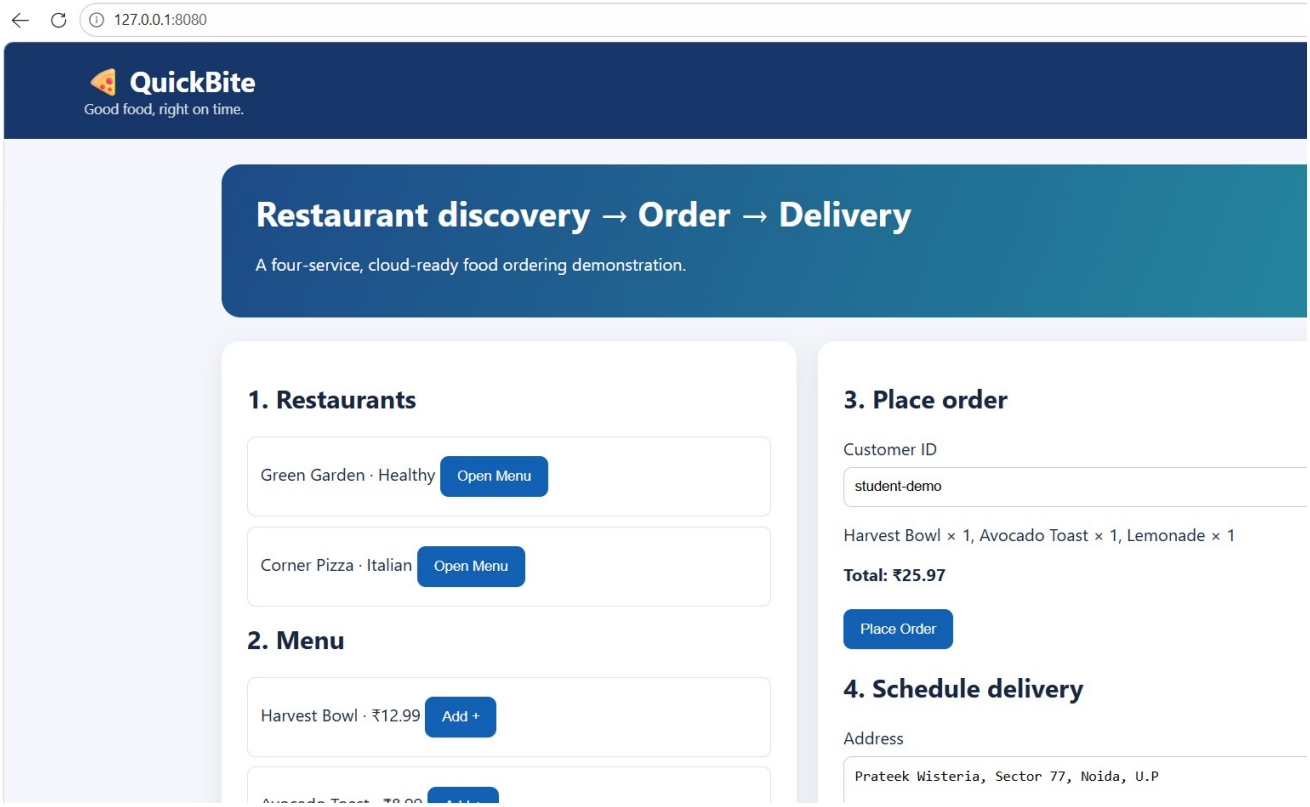
Flow step	Evidence and interpretation
Browse and select	A17–A18 show the restaurant list and menu for r2. The UI is shown in A25.

Flow step	Evidence and interpretation
Place and retrieve order	A19–A20 show HTTP 201 and the matching order lookup. The order ID is the reference passed into the delivery request.
Create and retrieve delivery	A22–A23 show successful delivery creation and lookup. A21 shows HTTP 409 when creation is repeated for the same order.
Scale Orders under CPU load	A34 and A39 show 2 → 8 → 2 replicas. HPA policy allows 2–10 replicas at 70% CPU. The recorded peak in those screenshots was 8.

A25 Customer Web UI

E25_QuickBite Web UI.png

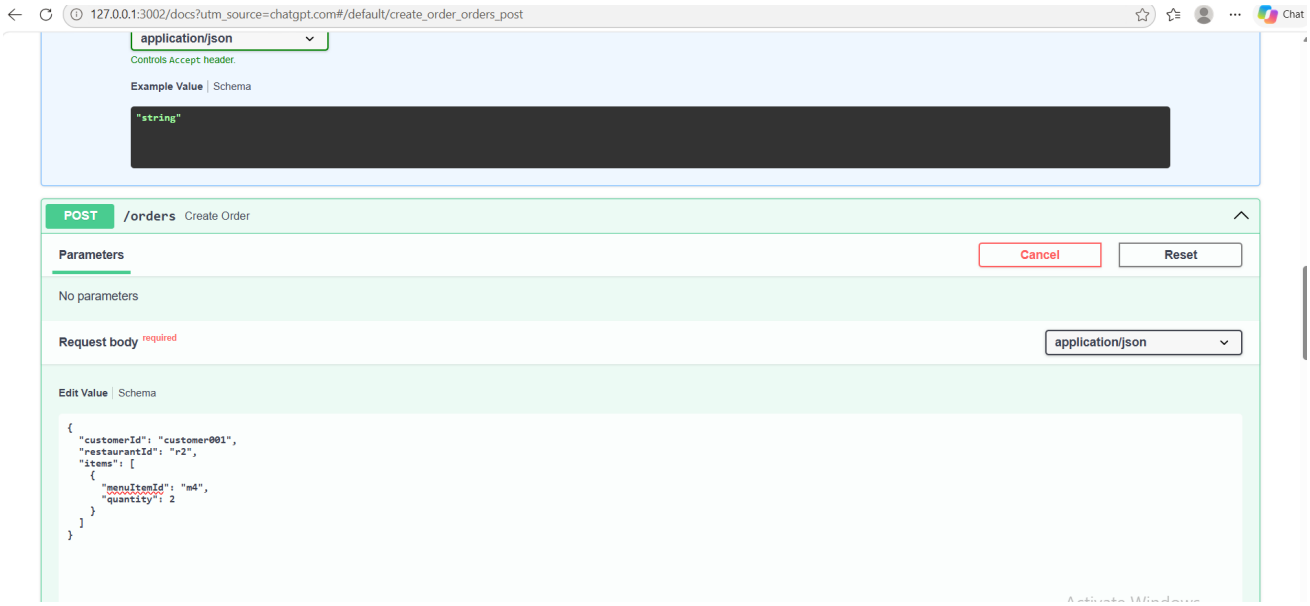
The QuickBite Web UI exposes the customer workflow used to browse and order through the gateway.



A27 Swagger order request schema

Swagger_Post.png

Swagger POST capture showing an API operation in the interactive documentation. A19 and A22 record the Postman outcomes.



3b QuickBite API testing with Postman

Operation through port 8080	Observed result	Screenshot
GET /api/catalog/restaurants	200 OK · restaurant list	E13
GET /api/catalog/restaurants/r2/menu	200 OK · Corner Pizza menu	E14
POST /api/orders/orders	201 Created · generated order ID · placed	E15
GET /api/orders/orders/{orderId}	Persisted order returned	E16
POST /api/deliveries/deliveries	201 Created · delivery status queued	E17
Repeat POST for same order	409 Conflict · duplicate rejected	E17A
GET /api/deliveries/deliveries/{deliveryId}	Delivery lookup captures, including retry evidence	E18
Browser UI and Swagger	Customer flow and API operations	E25 and Swagger captures

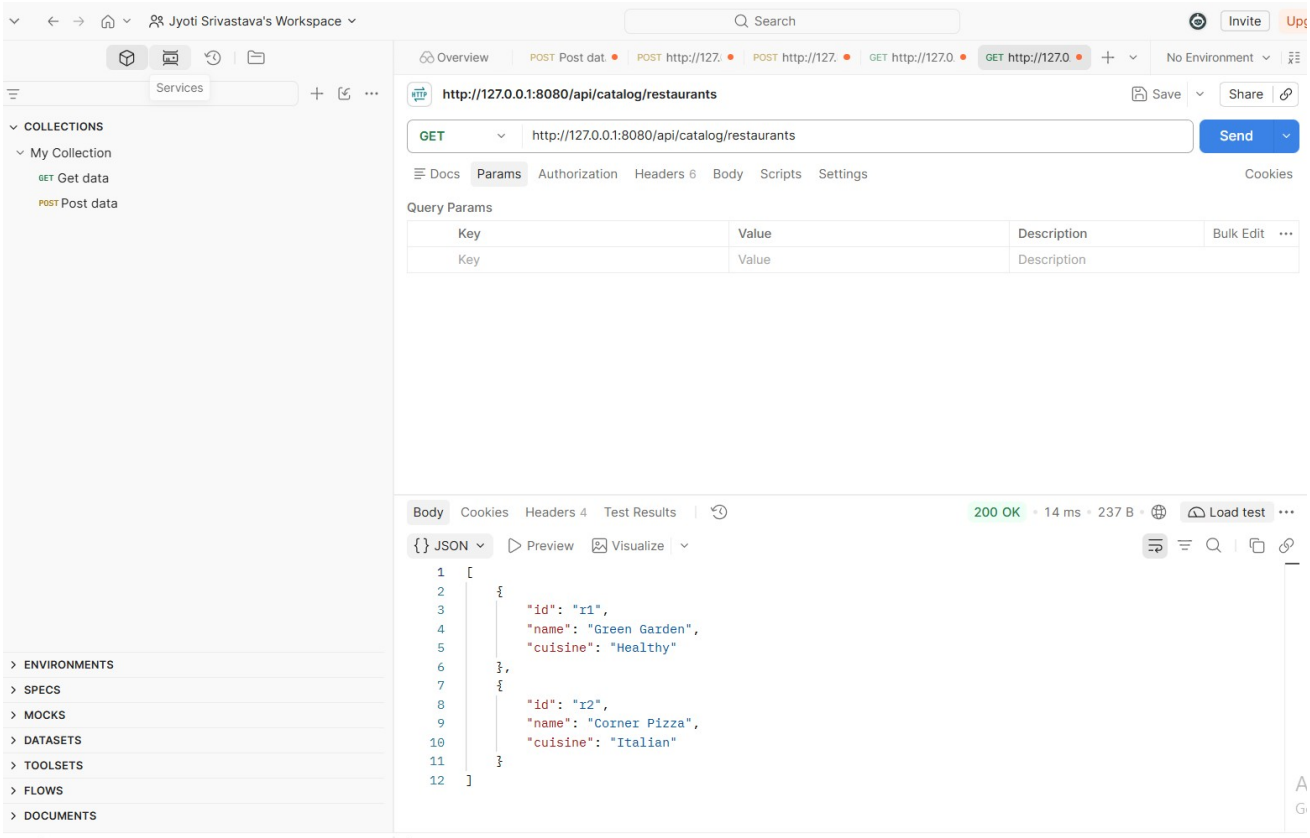
POST order
{"customerId":"customer001","restaurantId":"r2",
"items":[{"menuItemId":"m4","quantity":2}]}
POST delivery {"orderId":"<returned order ID>",
"customerAddress":"<demo delivery address>"}

Import postman/QuickBite.postman_collection.json and set baseUrl to http://127.0.0.1:8080. GET restaurants, GET the r2 menu, POST an order, GET its returned order ID, POST delivery for that ID, and GET the returned delivery ID. Repeating the delivery POST should return 409. The included Postman exhibits show the recorded responses.

A17 Restaurant list response

E13_Postman_Get_Restaurants.png

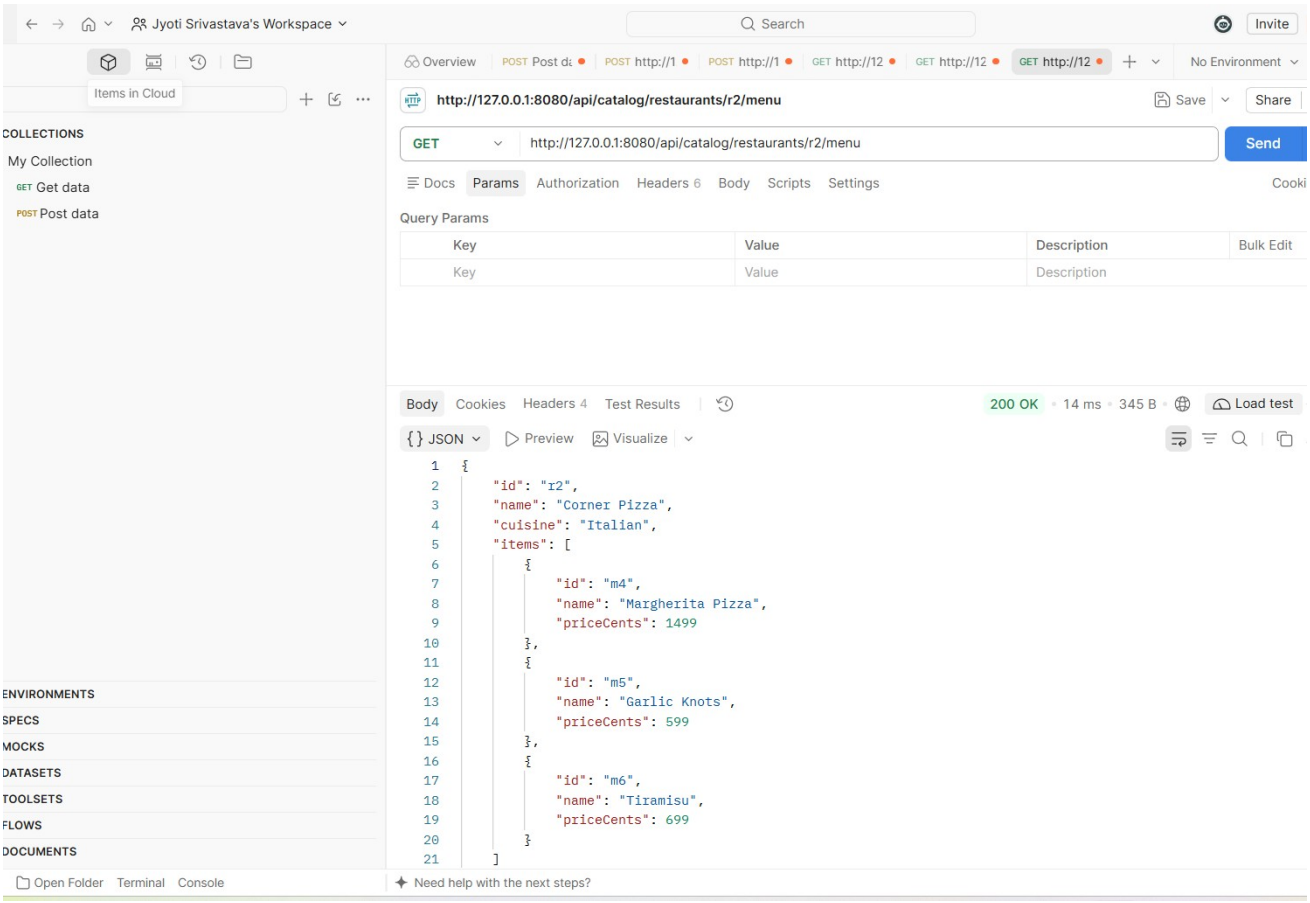
Postman sends the restaurant list GET through the gateway and receives **HTTP 200** with restaurant data.



A18 Restaurant menu response

E14_Postman_Restaurant_Menu.png

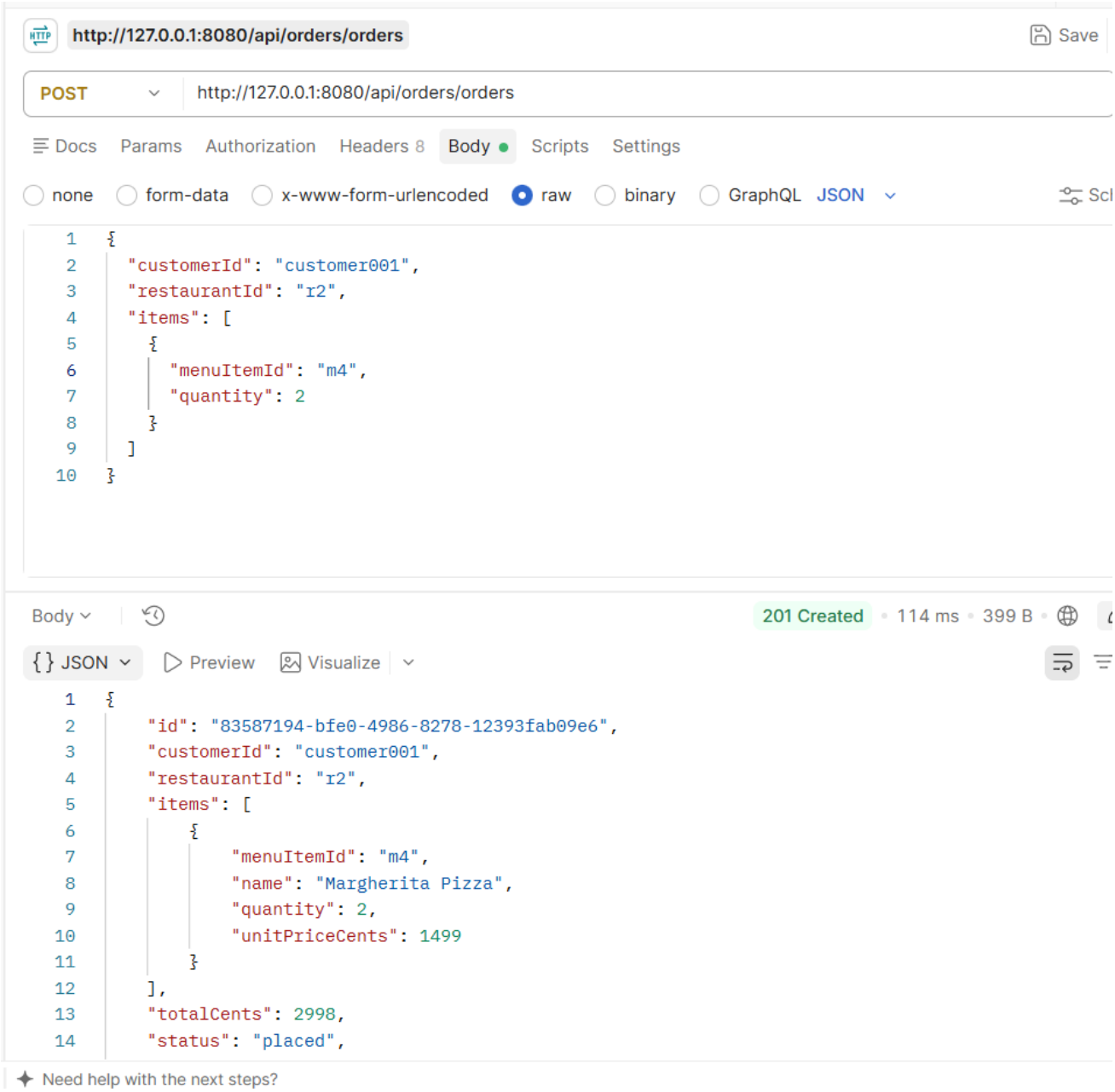
Postman retrieves the r2 menu. The returned menu identifies items that can be used in an order request.



A19 Order creation response

E15_Postman_Create_Order.png

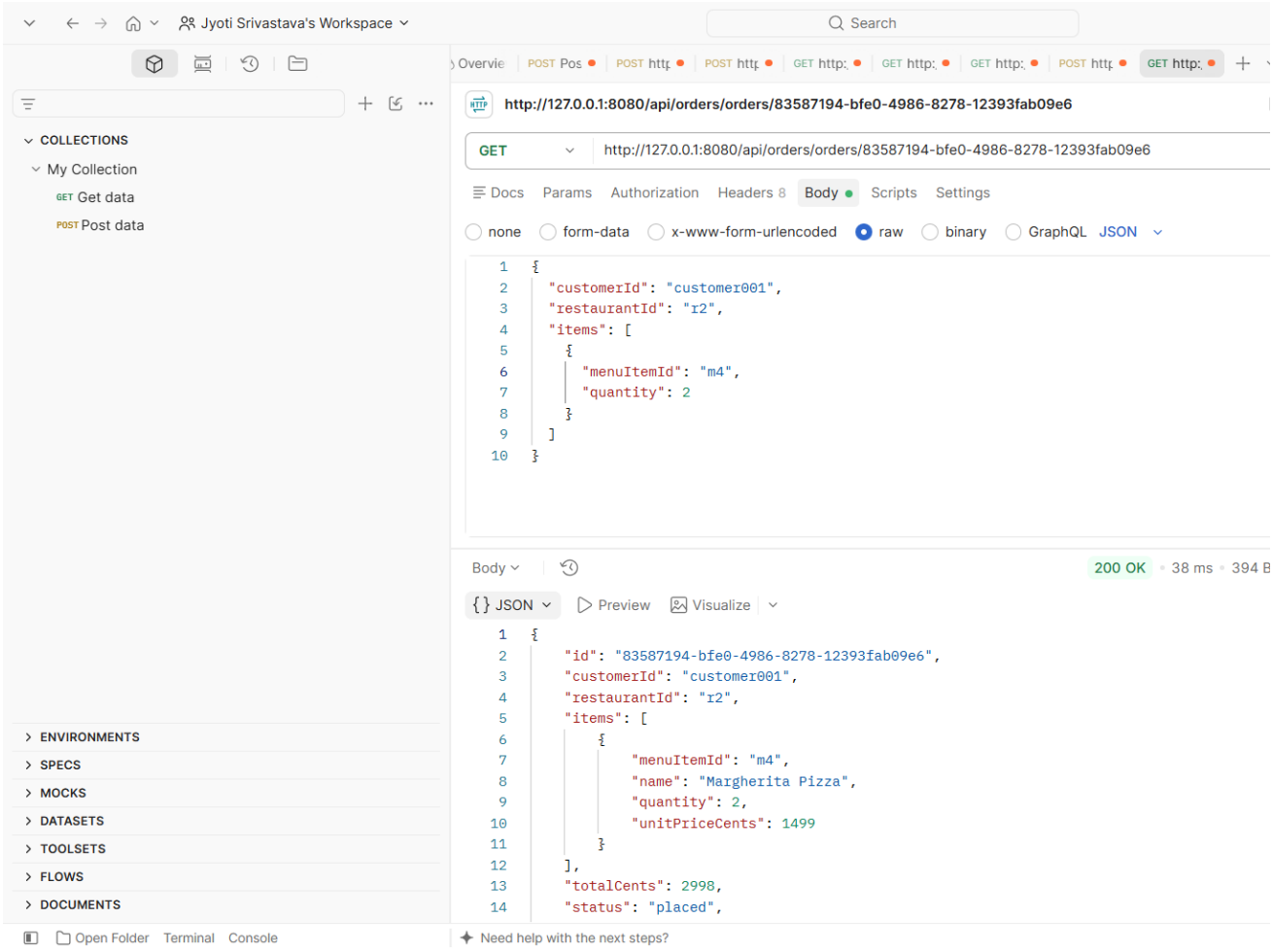
Postman submits an order and receives **HTTP 201** with an order ID and **status placed**.



A20 Persisted order lookup

E16_Postman_Get_Order_By_ID.png

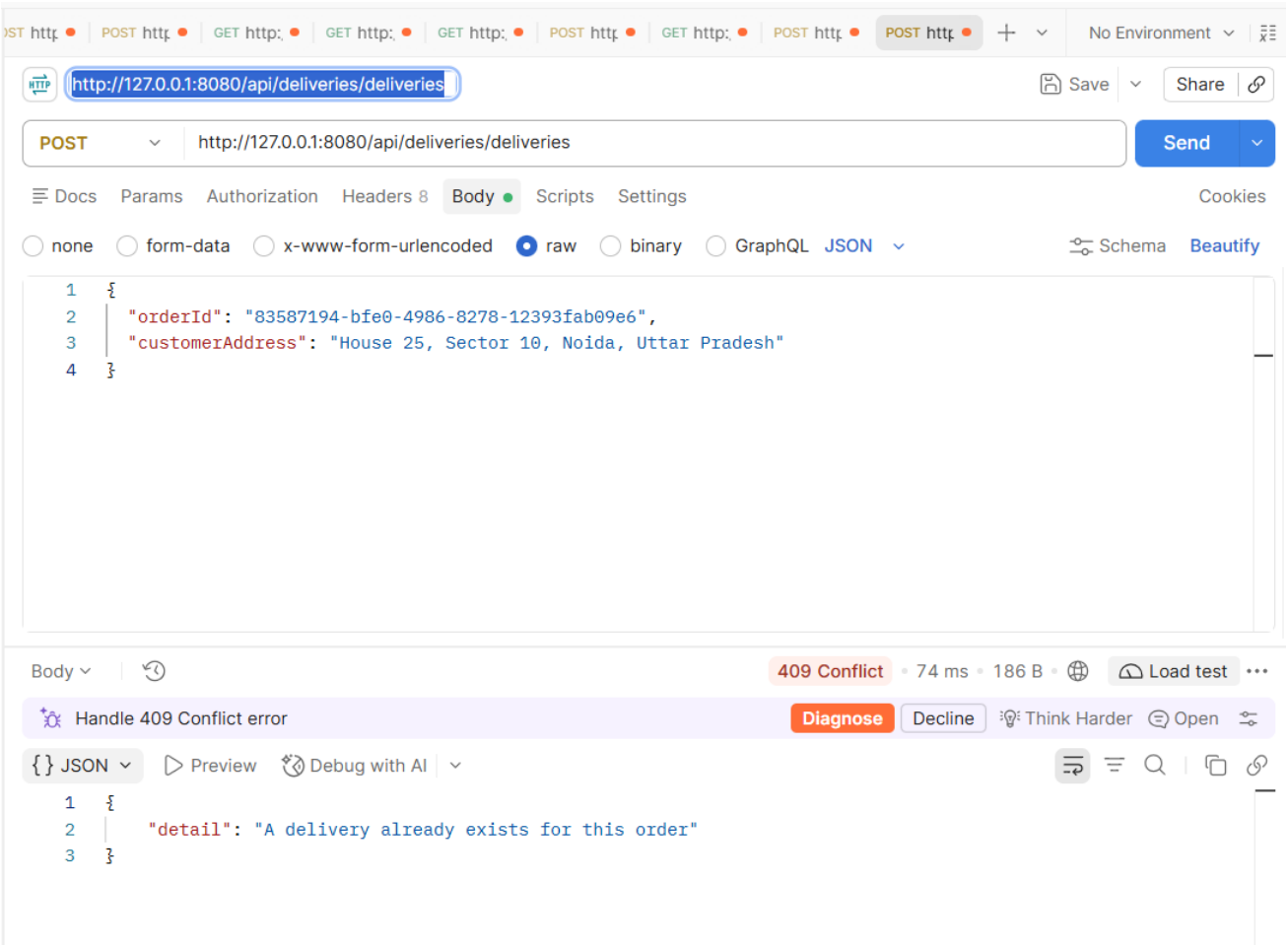
The order lookup retrieves the persisted order by its identifier, linking the response to A19.



A21 Duplicate delivery conflict

E17A_Duplicate_Delivery_409.png

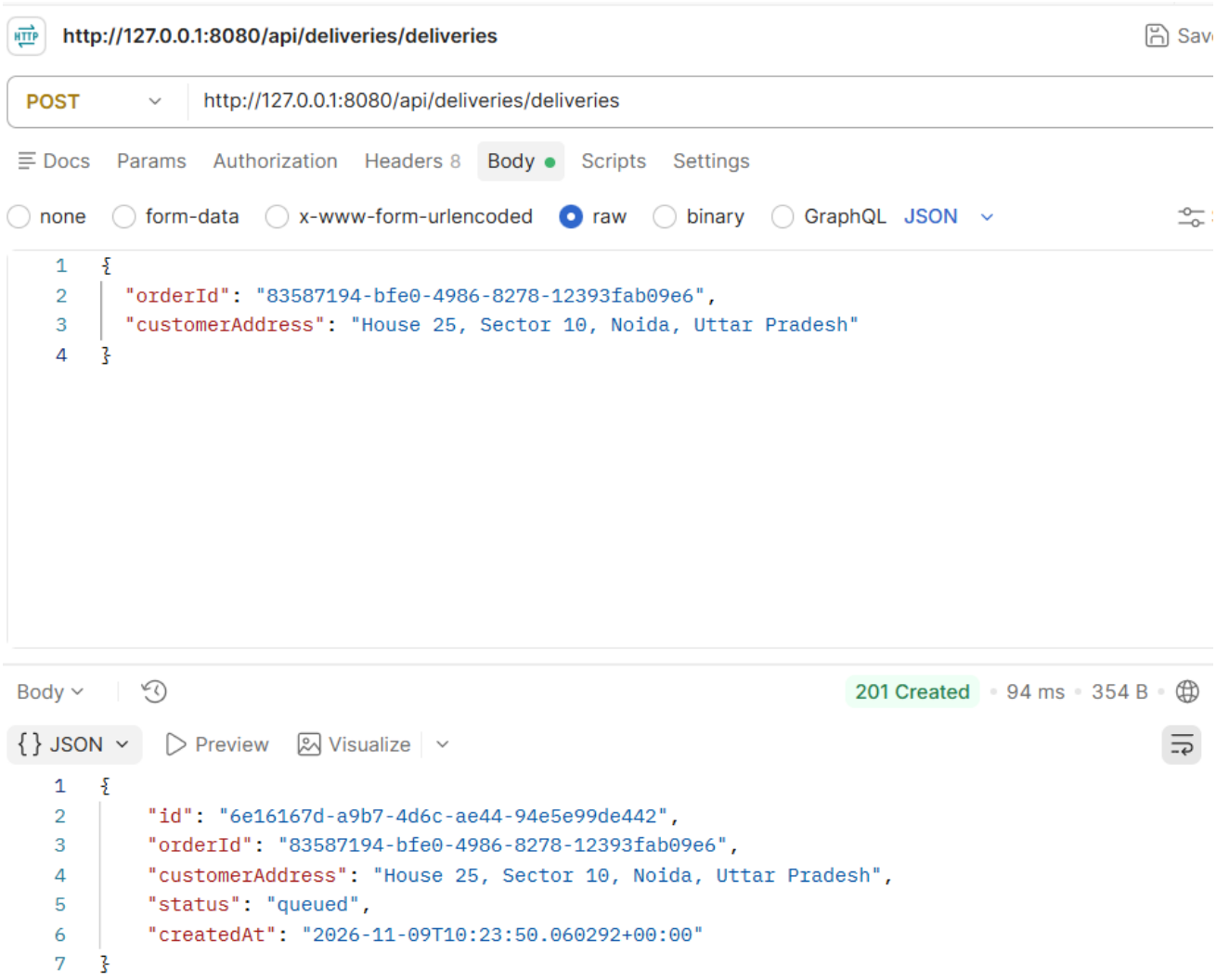
The repeated delivery POST returns **HTTP 409** for an order that already has a delivery record.



A22 Delivery creation response

E17_Postman_Create_Delivery.png

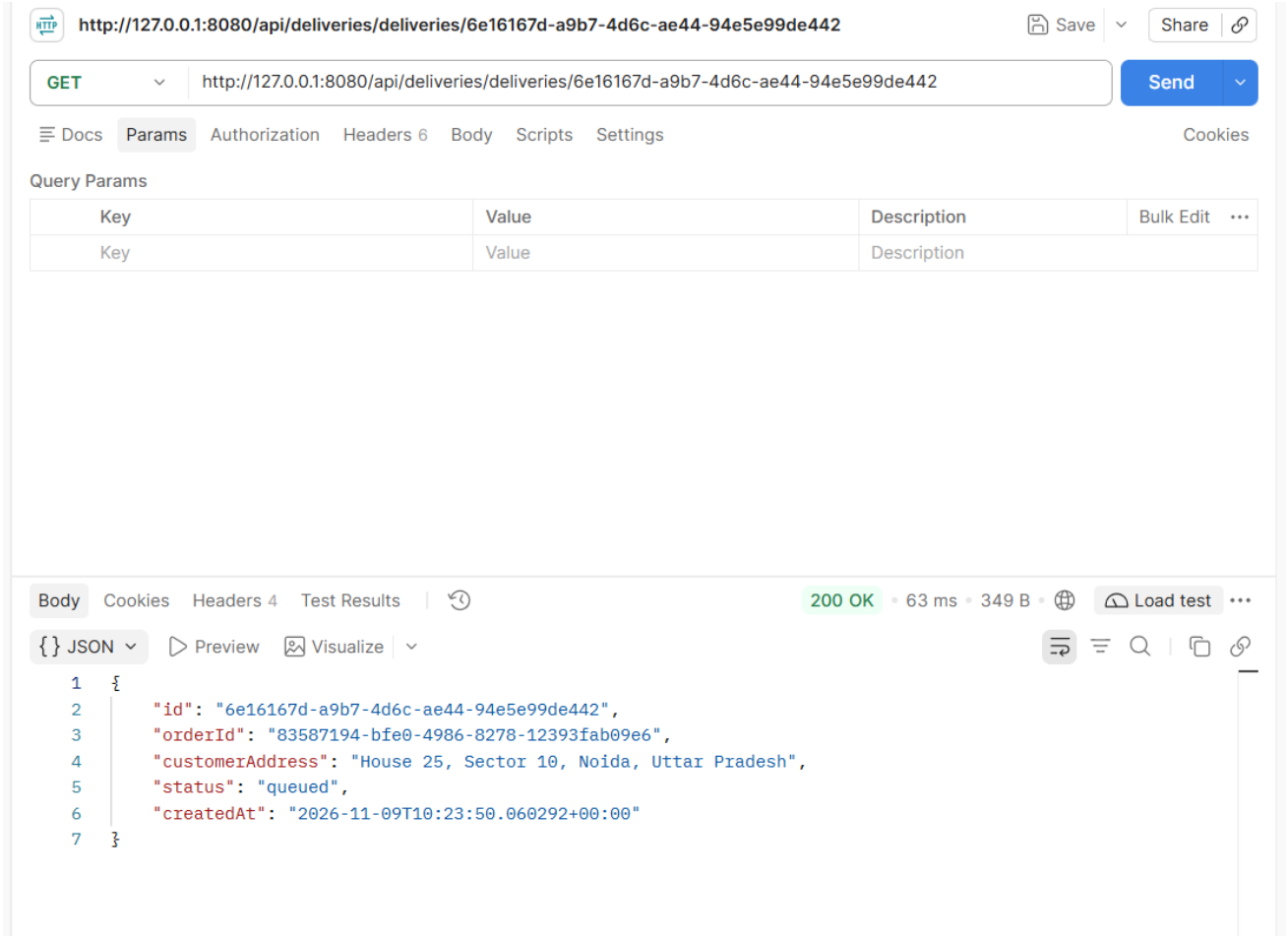
The first delivery POST returns **HTTP 201** and a delivery record with initial **status queued**.



A23 Persisted delivery lookup

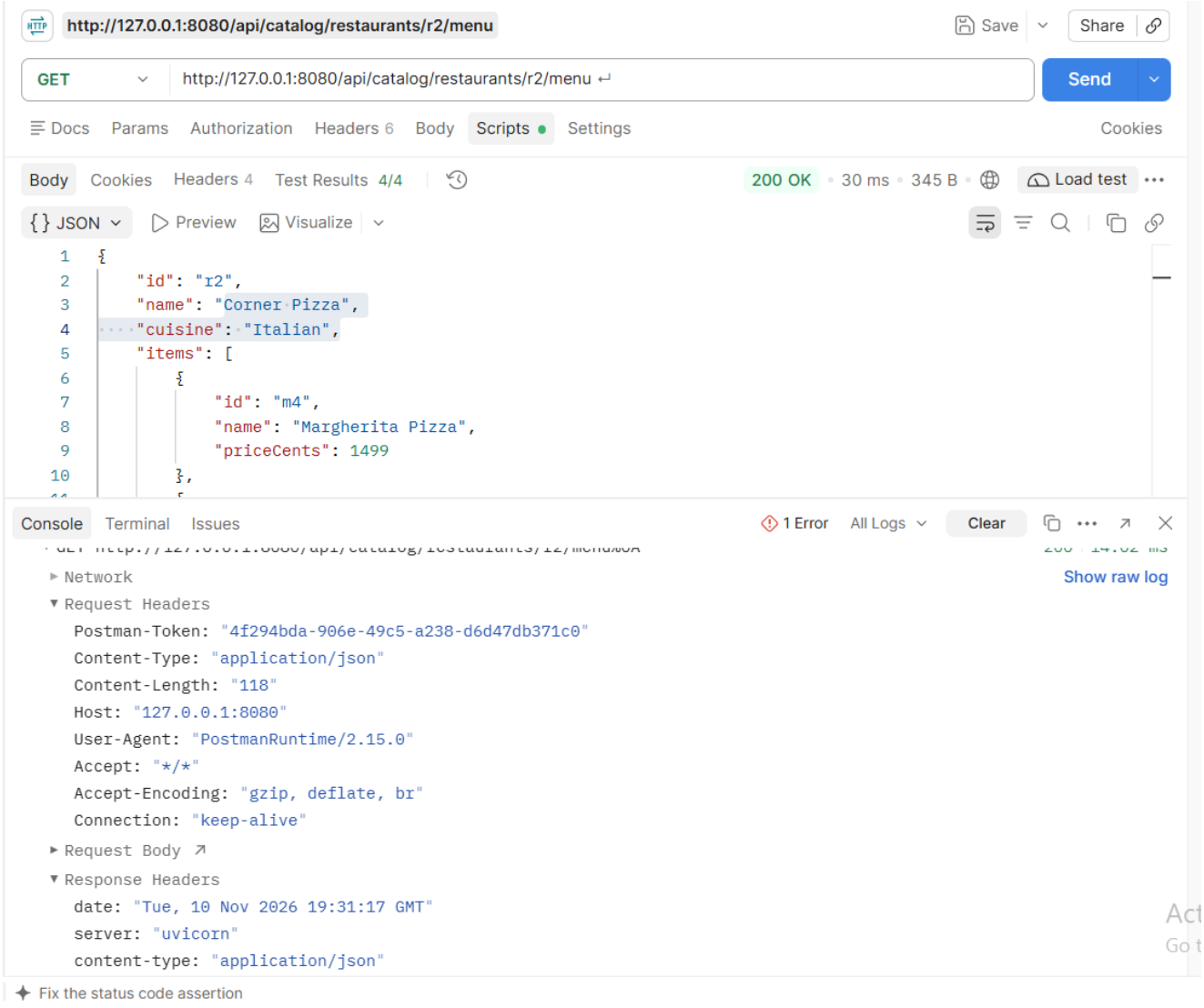
E18_!_Postman_Get_Delivery.png

The delivery lookup returns **HTTP 200** and the delivery record, including its queued status.



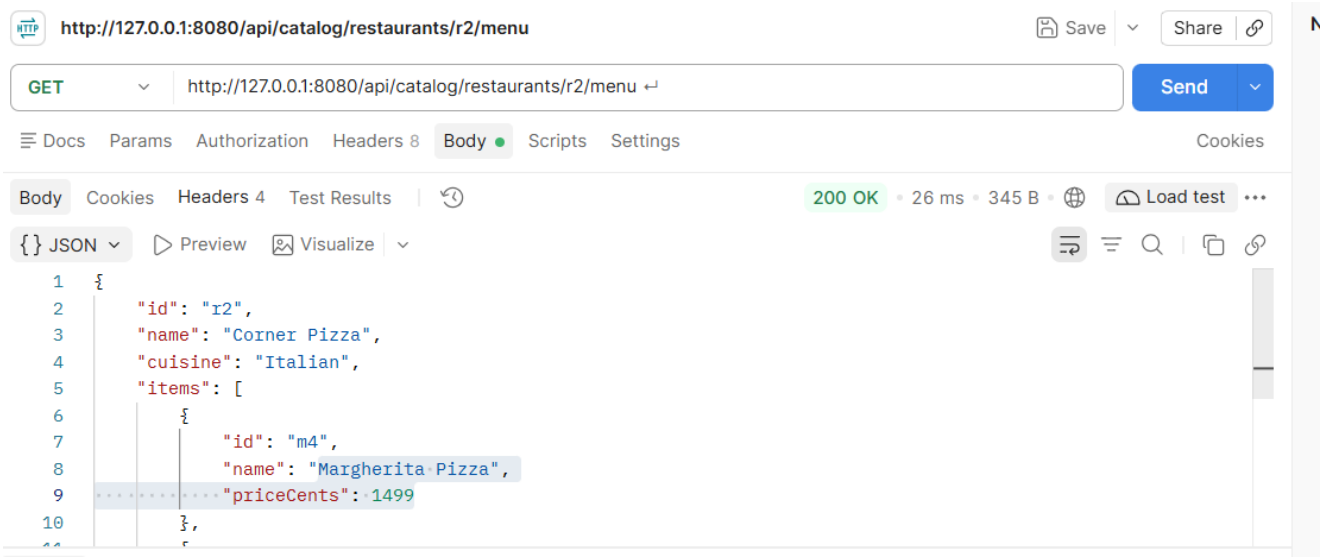
A60 Menu retrieval and Postman test results

GET /api/catalog/restaurants/r2/menu returns 200 OK (30 ms, 345 B). The response identifies restaurant r2 as Corner Pizza and includes item m4, Margherita Pizza, with priceCents 1499. The Test Results tab displays 4/4 for this request. A separate console error indicator is visible; its cause is not established by this capture.



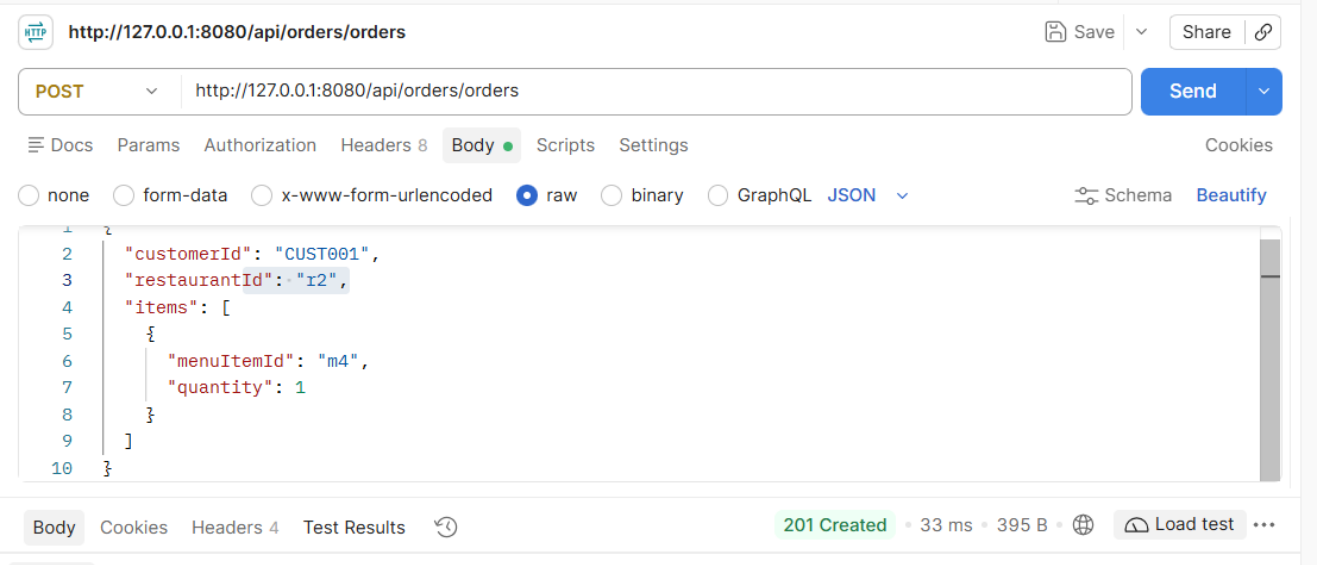
A61 Catalog menu response details

This response view shows GET /api/catalog/restaurants/r2/menu returning 200 OK (26 ms, 345 B). The JSON contains the restaurant identifier, name, cuisine and menu items. It demonstrates menu retrieval through the gateway at port 8080 and supplies the restaurant and menu-item identifiers used in the subsequent order request.



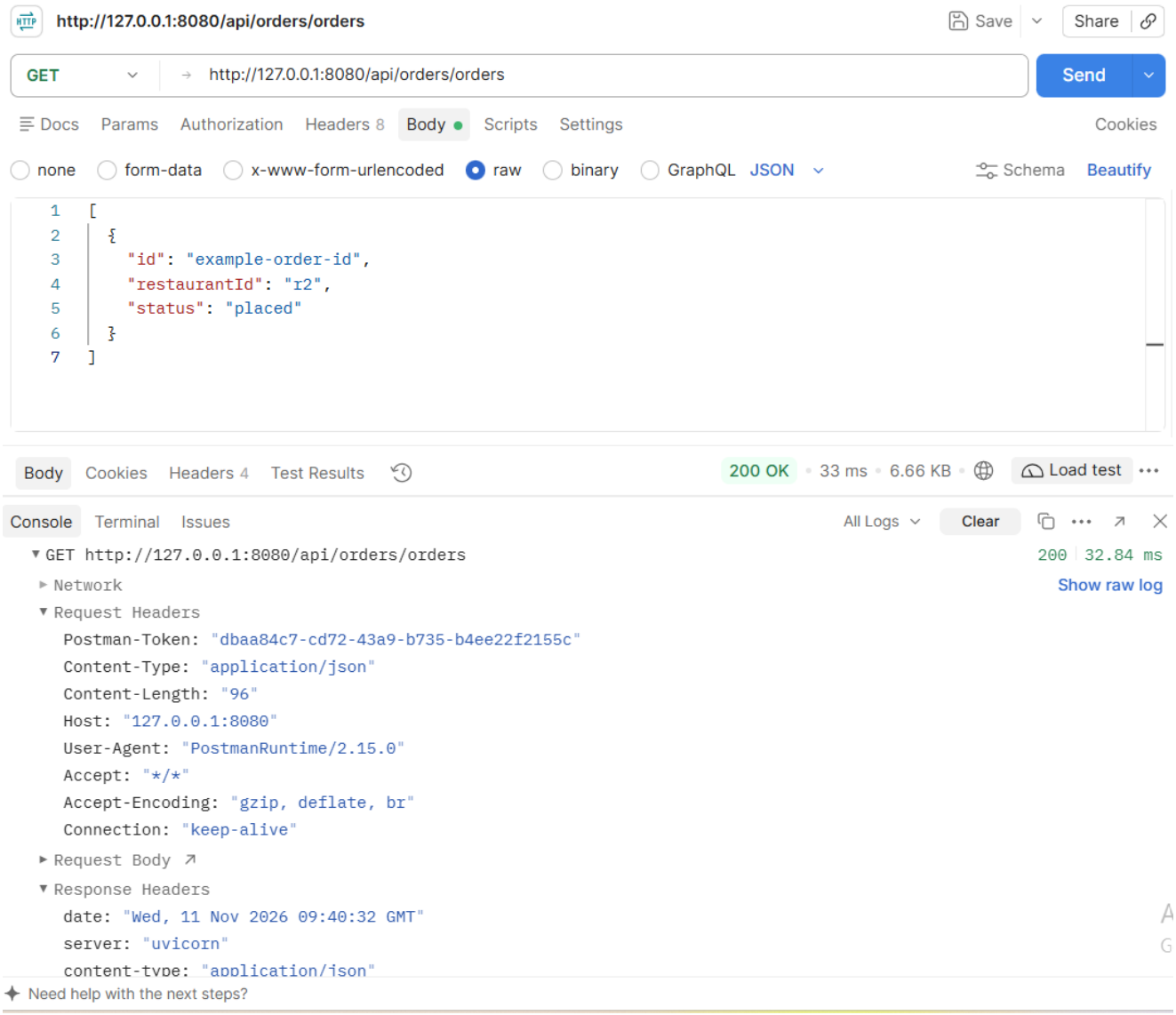
A62 Order creation request and success status

POST /api/orders/orders submits customerId CUST001, restaurantId r2 and one unit of menuItemId m4 as raw JSON. The displayed 201 Created status (33 ms, 395 B) confirms a successful creation response. This capture shows the submitted payload and status; the returned order identifier and response JSON are not visible here.



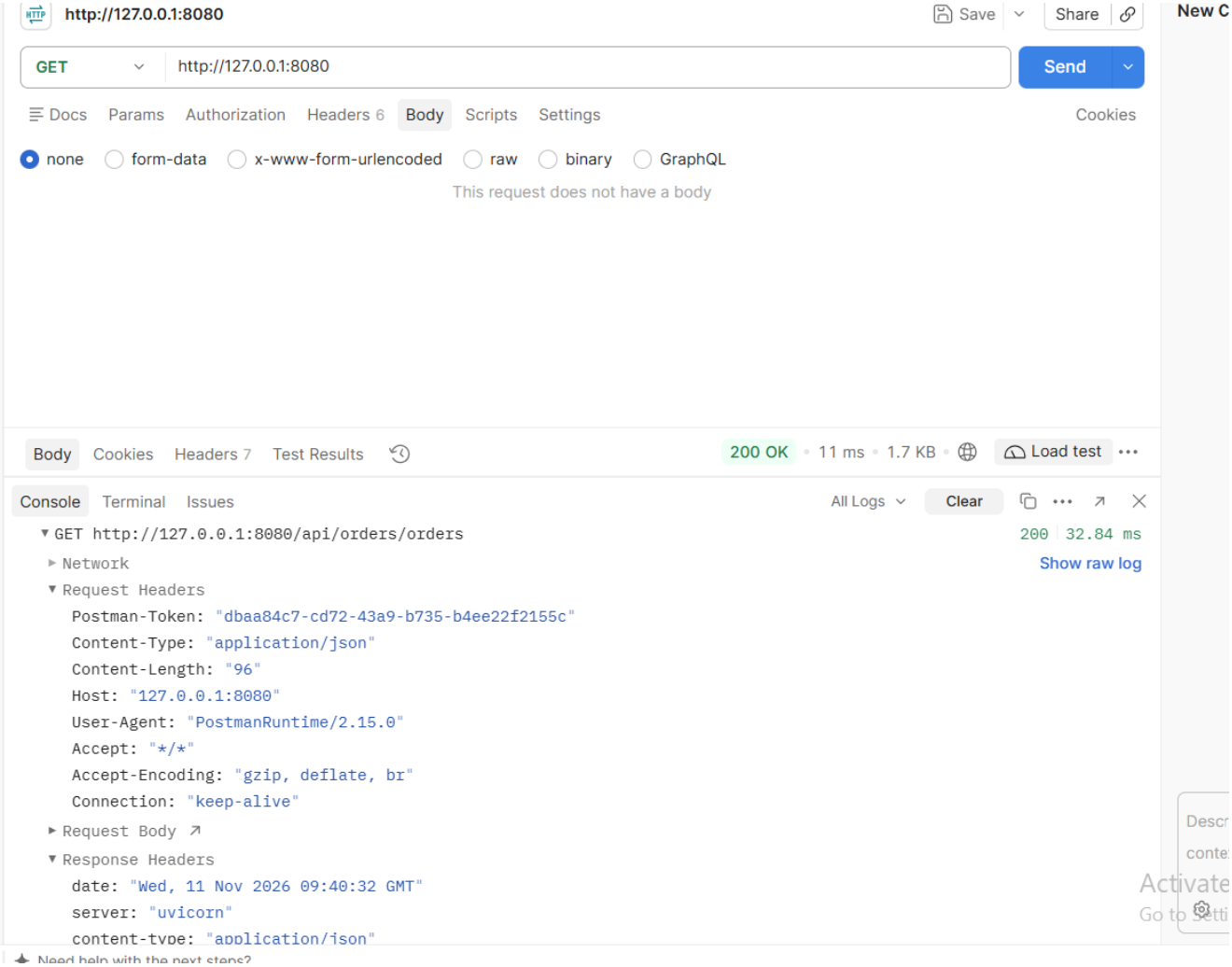
A63 Orders retrieval status and console trace

GET /api/orders/orders returns 200 OK (33 ms, 6.66 KB), and the console records the same endpoint with status 200. The JSON containing example-order-id is in the request Body editor, not the service response. For this GET operation, Body should be set to None. This image supports successful endpoint access; the earlier order-response evidence supplies the returned record details.



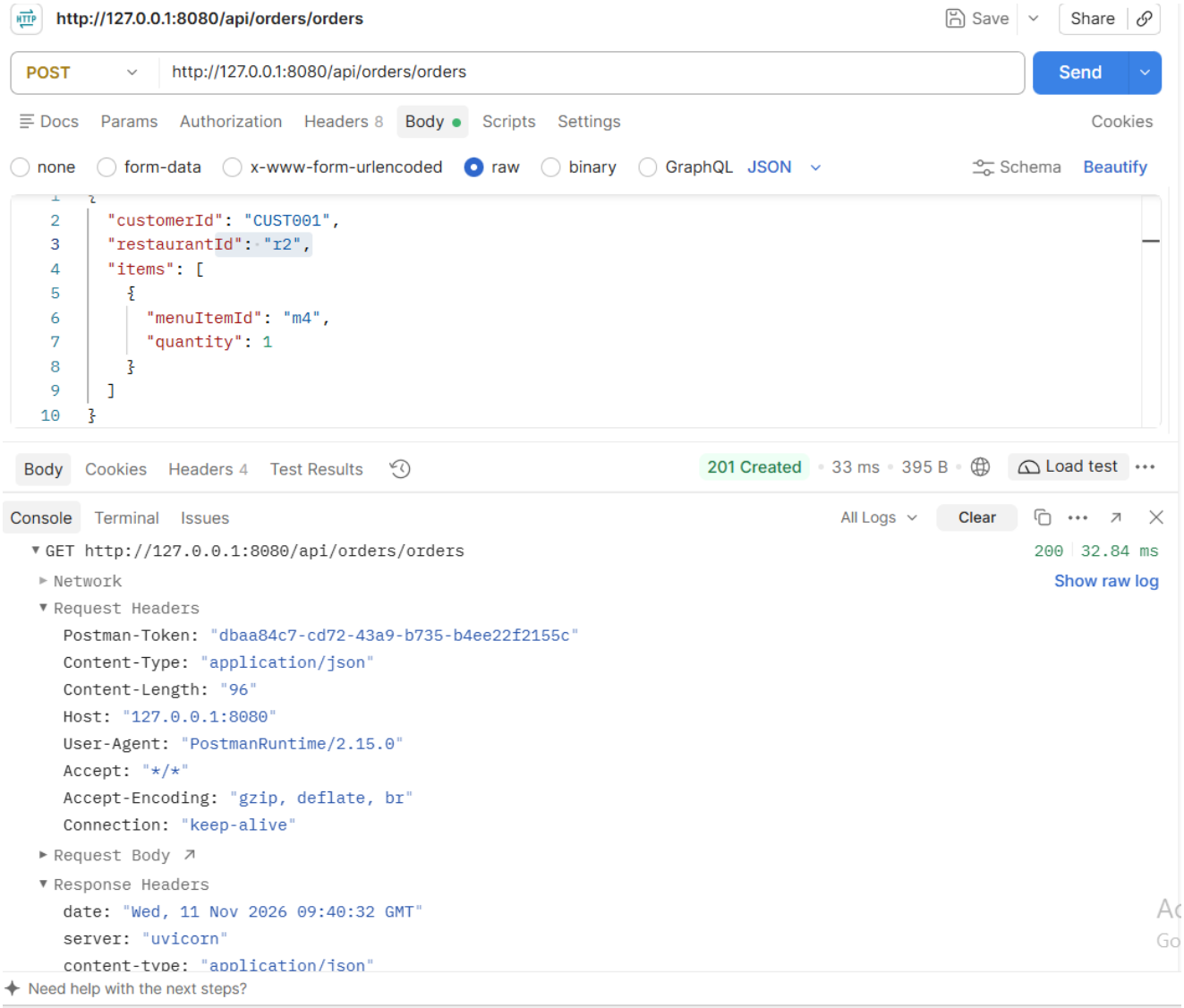
A64 Gateway root endpoint availability

GET http://127.0.0.1:8080 uses Body None and returns 200 OK (11 ms, 1.7 KB), demonstrating that the gateway root endpoint responds. The lower console retains an earlier GET /api/orders/orders entry, so that entry must not be attributed to this root request. The root response body is not expanded in this screenshot.



A65 Order creation with retained console context

The current POST /api/orders/orders shows the CUST001, r2 and m4 payload and a 201 Created response (33 ms, 395 B). The lower console still displays the earlier GET orders call, not the current POST. This additional capture preserves the original test context and complements A62; it does not independently show a new order identifier or a second creation event.



4 Implementation and project structure

Python and FastAPI expose typed JSON APIs and OpenAPI documentation. HTTP REST provides a uniform interface for the gateway and clients. PostgreSQL stores order and delivery records in separate databases, and Catalog serves its own restaurant data. The UI uses the gateway as its browser entry point.

Implementation steps

1. Implement Catalog GET endpoints for restaurants and menus using the service dataset. 2. Define Pydantic request models and SQLAlchemy records in Orders and Delivery. 3. Have Orders validate menu items against Catalog, calculate totals, and commit the order. 4. Have Delivery validate the order with Orders and enforce one delivery per order through a database constraint. 5. Expose the UI and REST forwarding through Gateway. 6. Package each service with its Dockerfile and dependencies, then verify the workflow in Postman and pytest before Kubernetes rollout.

The integration repository contains four service folders, Kubernetes manifests, tests and a GitHub Actions workflow. The structures below were inspected directly at commit ce8c3f6831ecdff891996415cfafe86724c6e47a. All four services are available under services/ in this single repository, each with its own source, Dockerfile and dependencies.

```
QuickBites/
  services/
    catalog/
      main.py              FastAPI and catalog dataset
      requirements.txt       Runtime dependencies
      Dockerfile            Catalog image and Uvicorn
entry point
      README.md
      tests/test_catalog_service.py
    orders/
      main.py              API, schemas, Order model
and DB session
      requirements.txt
      Dockerfile
      README.md
      tests/test_orders_service.py
    delivery/
      main.py              API, schemas, Delivery model
and DB session
      requirements.txt
      Dockerfile
      README.md
      tests/test_delivery_service.py
    gateway/
      main.py              HTTP proxy, health checks and
UI entry point
      requirements.txt
```

Dockerfile
README.md
tests/test_gateway_service.py
static/index.html
static/app.js
static/style.css
deploy/kubernetes/ 00-namespace.yaml through 08-pdb.yaml
.github/workflows/quickbite-ci.yml
postman/QuickBite.postman_collection.json
tests/test_catalog.py
tests/test_order_validation.py
scripts/smoke_test.py
scripts/load_test.py
docker-compose.yml
requirements-dev.txt
run_local.py
README.md

Service entry points and data ownership

Each Dockerfile starts Uvicorn with main:app. Catalog listens on 3001, Orders on 3002, Delivery on 3003, and Gateway on 8080. Orders and Delivery define their SQLAlchemy models, engine and session setup inside main.py. Orders stores the accepted item snapshot and server-calculated total. Delivery stores a separate delivery ID, order ID, address, status and timestamp, with a unique constraint on order_id.

5 Local execution and Docker deployment

Docker Compose runs four independently packaged application containers and two PostgreSQL containers. The gateway is published at 127.0.0.1:8080 for local use. In Minikube, the four Kubernetes Services expose the application components within food-delivery, while the database workloads use persistent volume claims.

Execution instructions

Prerequisites and local verification

Use PowerShell in the integration repository root with Docker Desktop running and Minikube available. These commands follow the inspected GitHub version. Images tagged :local match the checked-in manifests. The CI/CD job uses commit-based tags when updating an existing deployment.

```
git clone
https://github.com/erjyotisrvtv-lab/QuickBites.git
cd QuickBites
docker compose config
docker compose up -d --build
docker compose ps
```

The next test commands use the running Compose gateway. Stop Compose before the Minikube port-forward so that localhost port 8080 is available.

After the Compose stack is healthy, run the project tests and the gateway smoke test in a Python environment with the

development dependencies. These are reproduction commands; the recorded test results are in A53.

```
python -m pip install -r requirements-dev.txt
python -m pytest tests/ -v
python scripts/smoke_test.py
```

```
docker compose stop
```

A30 Service Docker images

E12_Docker_Images.png

The image listing shows the container packages available for the services.

```
PS C:\Users\jyoti.s07\Desktop\QuickBite> & "$env:LOCALAPPDATA\Programs\DockerDesktop\resources\bin\docker.exe" images
                                                                                                    i Info →  U  In Use
IMAGE                                                                                ID             DISK USAGE   CONTENT SIZE   EXTRA
gcr.io/k8s-minikube/kicbase:v0.0.51                                                  146cd636030e       1.89GB          532MB
gcr.io/k8s-minikube/kicbase@sha256:4a1c825b61479e6c898851ea66f13c620aaeab6002746e95067fc2c4b38a0b24  4a1c825b6147       1.89GB          532MB    U
postgres:16-alpine                                                                   aa6cb33cf4c1       420MB           117MB    U
quickbite-catalog:ci                                                                 333b2b42a64c       202MB          48.8MB
quickbite-catalog:latest                                                             c59c9b13ddfa       202MB          48.8MB    U
quickbite-delivery:latest                                                            8d0a7fe7bb24       258MB            62MB    U
quickbite-gateway:latest                                                             2d527abcc27a       204MB          49.4MB    U
quickbite-orders:latest                                                              9c5c7645fcde       258MB            62MB    U
quickbite/catalog:ce8c3f6831ecdff891996415cfafe86724c6e47a                           f58afb716525       202MB          48.8MB
quickbite/catalog:local                                                              c59c9b13ddfa       202MB          48.8MB    U
quickbite/delivery:ce8c3f6831ecdff891996415cfafe86724c6e47a                          b0b2e66a872d       258MB            62MB
quickbite/delivery:local                                                             8d0a7fe7bb24       258MB            62MB    U
quickbite/gateway:ce8c3f6831ecdff891996415cfafe86724c6e47a                           1a288fe135cf       204MB          49.4MB
quickbite/gateway:local                                                              2d527abcc27a       204MB          49.4MB    U
quickbite/orders:ce8c3f6831ecdff891996415cfafe86724c6e47a                            55a2139fdf9e       258MB            62MB
quickbite/orders:local                                                               9c5c7645fcde       258MB            62MB    U
```

A31 Healthy Docker Compose stack

E35_Docker_Compose_Services.png

Compose status records the gateway, three business services and two PostgreSQL containers in the local stack.

```
PS C:\Users\jyoti.s07\Desktop\QuickBite> & "$env:LOCALAPPDATA\Programs\DockerDesktop\resources\bin\docker.exe" compose ps
NAME                    IMAGE                                                                COMMAND                  SERVICE       CREATED         STATUS
          PORTS
quickbite-catalog-1     sha256:91cdc6c598d4f401cb7300969435f0f6a3a8f4e24eda92565375d25d6bfe11f7   "uvicorn main:app --…"   catalog       29 hours ago    Up 25 hours
(healthy)   3001/tcp
quickbite-delivery-1    sha256:fac4db88f52d717441160f9ebbf473eaad83d0a762331f896b54ac8745d73bc9   "uvicorn main:app --…"   delivery      29 hours ago    Up 25 hours
(healthy)   3003/tcp
quickbite-delivery-db-1 postgres:16-alpine                                                   "docker-entrypoint.s…"   delivery-db   29 hours ago    Up 25 hours
(healthy)   5432/tcp
quickbite-gateway-1     sha256:13a8370d399a69e83d5a16e2ff893b1528c02ffceb68e372849c55df6b109fdd   "uvicorn main:app --…"   gateway       29 hours ago    Up 25 hours
(healthy)   127.0.0.1:8080->8080/tcp
quickbite-orders-1      sha256:8bf3c267a29cf7731db1ad4b9968c70874bc370fb0d5f10c390d8ba1057cad8c   "uvicorn main:app --…"   orders        29 hours ago    Up 25 hours
(healthy)   3002/tcp
quickbite-orders-db-1   postgres:16-alpine                                                   "docker-entrypoint.s…"   orders-db     29 hours ago    Up 25 hours
```

6 Kubernetes deployment

Deployment stage	Plan and verification
1 Package and smoke test	Build service images, start the Compose stack and confirm container health, UI and REST operations. E12 and E35.
2 Prepare cluster and storage	Start Minikube, enable metrics, create the namespace, and apply the database manifests before dependent application workloads.
3 Deploy services	Apply Catalog, Orders, Delivery and Gateway workloads and Services. Confirm each application Deployment reaches 2/2 ready replicas.
4 Verify persistence	Check orders-db-0 and delivery-db-0 and the two Bound claims. E10 records PVC status.
5 Inspect dashboard	Select food-delivery and inspect Deployments and Pods. E05 and E06 corroborate the CLI workload snapshots.

Deployment stage	Plan and verification
6 Apply scaling and disruption policies	Verify the four HPAs, four PDBs and resource metrics. Watch Orders replicas during and after CPU stress.

Build and load images

```
minikube start --driver=docker
minikube addons enable metrics-server
docker build -t quickbite/catalog:local
./services/catalog
docker build -t quickbite/orders:local
./services/orders
docker build -t quickbite/delivery:local
./services/delivery
docker build -t quickbite/gateway:local
./services/gateway
minikube image load quickbite/catalog:local
minikube image load quickbite/orders:local
minikube image load quickbite/delivery:local
minikube image load quickbite/gateway:local
```

Apply manifests in dependency order

```
minikube kubectl -- apply -f deploy/kubernetes/00-
namespace.yaml
minikube kubectl -- apply -f deploy/kubernetes/01-
secret-demo.yaml
minikube kubectl -- apply -f deploy/kubernetes/02-
```

```
databases.yaml
minikube kubectl -- rollout status statefulset/orders-db -n food-delivery --timeout=180s
minikube kubectl -- rollout status statefulset/delivery-db -n food-delivery --timeout=180s
minikube kubectl -- apply -f deploy/kubernetes/03-catalog.yaml
minikube kubectl -- apply -f deploy/kubernetes/04-orders.yaml
minikube kubectl -- apply -f deploy/kubernetes/05-delivery.yaml
minikube kubectl -- apply -f deploy/kubernetes/06-gateway.yaml
minikube kubectl -- apply -f deploy/kubernetes/07-hpa.yaml
minikube kubectl -- apply -f deploy/kubernetes/08-pdb.yaml
```

The Secret supplies database connection settings. The database manifest creates two StatefulSets, Services and persistent volume claims. The application manifests configure Service discovery, health probes and CPU requests. The HPA and PDB manifests apply the scaling and disruption policies to all four application Deployments.

Access and inspect

```
minikube kubectl -- rollout status
deployment/catalog -n food-delivery --timeout=180s
minikube kubectl -- rollout status
deployment/orders -n food-delivery --timeout=180s
minikube kubectl -- rollout status
deployment/delivery -n food-delivery --
timeout=180s
minikube kubectl -- rollout status
deployment/gateway -n food-delivery --
timeout=180s
minikube kubectl -- get
deploy,statefulset,pods,svc,pvc,hpa,pdb -n food-
delivery
minikube kubectl -- top pods -n food-delivery
minikube kubectl -- port-forward service/gateway
8080:8080 -n food-delivery
```

Keep the port-forward terminal open and browse http://127.0.0.1:8080. Use http://127.0.0.1:8080/docs for gateway Swagger. In another terminal, run the Dashboard command below and select food-delivery to inspect Deployments, Pods, StatefulSets and Services.

```
minikube addons enable dashboard
minikube dashboard
```

Runtime configuration

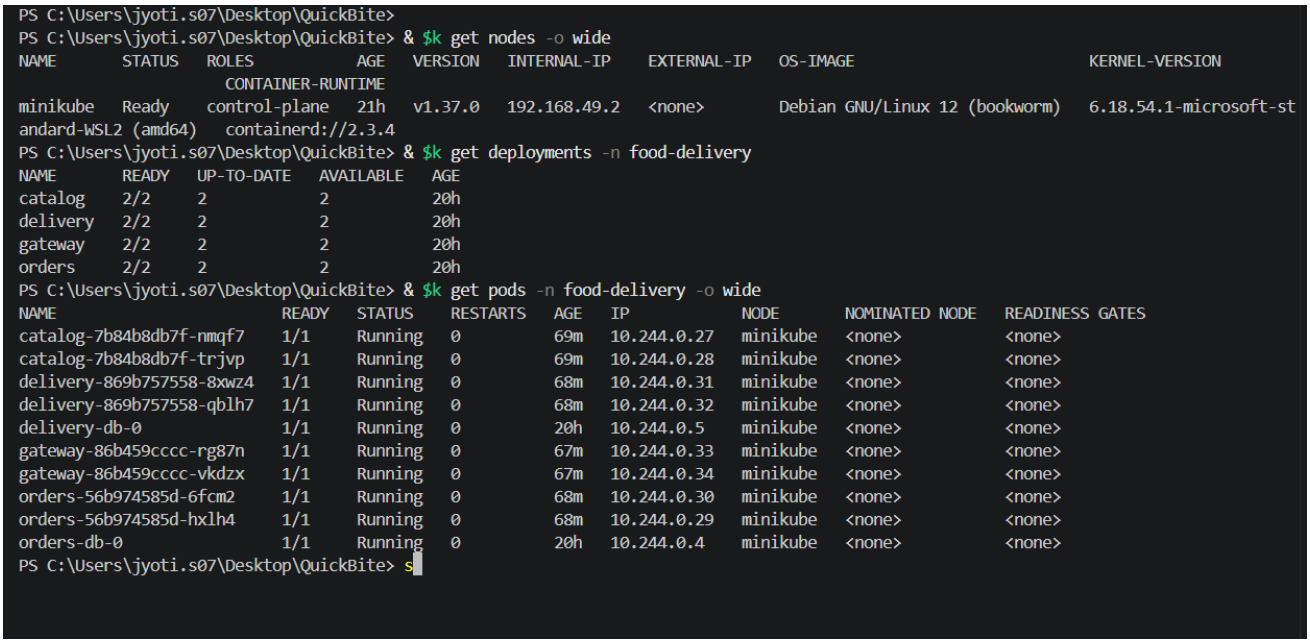
Parameter	Configured value	Evidence or source
Application baseline	2 replicas each for Catalog, Orders, Delivery and Gateway	A09 and A11
Service ports	Catalog 3001; Orders 3002; Delivery 3003; Gateway 8080	A10
CPU request and limit	100m requested; 500m limit per application container	03-catalog.yaml to 06-gateway.yaml
Memory request and limit	96Mi requested; 256Mi limit per application container	Application manifests
HPA target and bounds	CPU 70%; minimum 2; maximum 10 for all four services	A13 and 07-hpa.yaml
PDB	minAvailable 1 for each application	A14 and 08-pdb.yaml
PostgreSQL storage	Two Bound PVCs, 1Gi each	A16 and 02-databases.yaml

Parameter	Configured value	Evidence or source
Health probes	/health readiness and /live liveness	Application manifests
Recorded Orders load test	2 → 8 → 2 replicas; CPU 252% at the captured peak	A34 and A39

A09 Cluster node, Deployments and pods

E03_Kubernetes_Pods_Running.png

The inventory shows the application Deployments and PostgreSQL pods in food-delivery. The Dashboard views in A11–A12 corroborate workload readiness.



A10 Kubernetes Service endpoints

E04_Kubernetes_Services.png

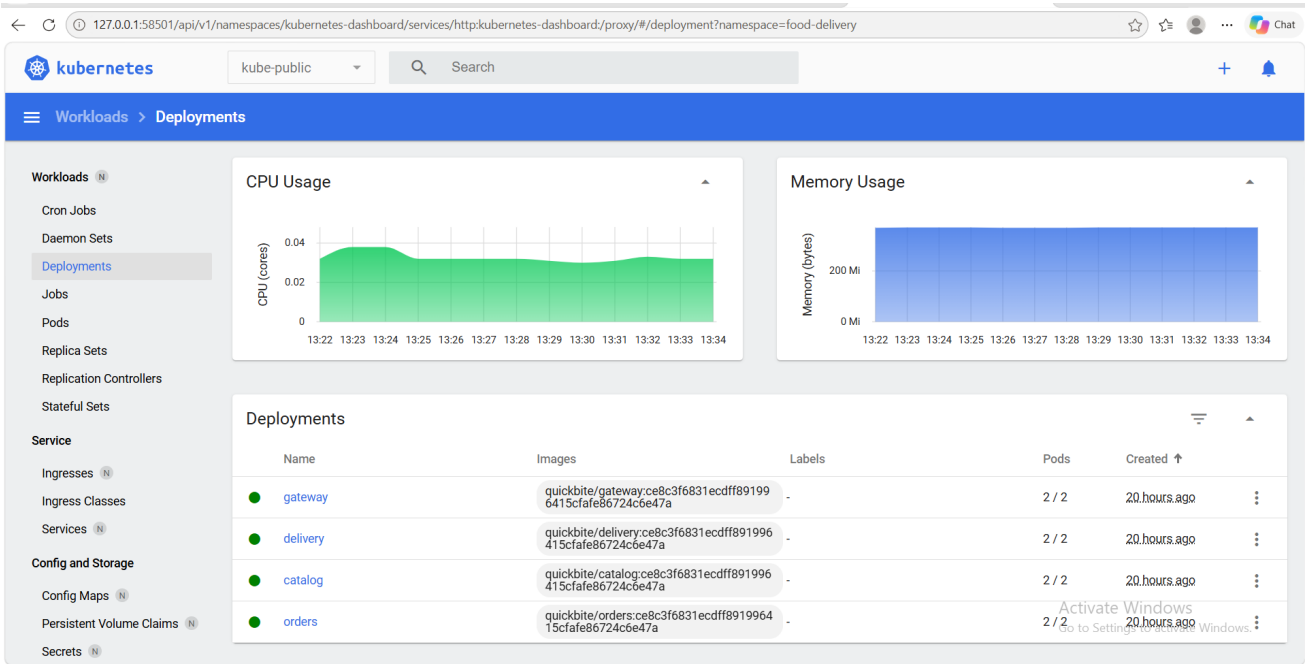
The Kubernetes Services listing shows the internal endpoints for the gateway, application services and database workloads.

```
PS C:\Users\jyoti.s07\Desktop\QuickBite>
PS C:\Users\jyoti.s07\Desktop\QuickBite> & $k get services -n food-delivery -o wide
NAME          TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)    AGE   SELECTOR
catalog       ClusterIP   10.110.82.73     <none>        3001/TCP   20h   app=catalog
delivery      ClusterIP   10.100.147.42    <none>        3003/TCP   20h   app=delivery
delivery-db   ClusterIP   10.102.122.92    <none>        5432/TCP   20h   app=delivery-db
gateway       ClusterIP   10.102.229.241   <none>        8080/TCP   20h   app=gateway
orders        ClusterIP   10.102.11.8      <none>        3002/TCP   20h   app=orders
orders-db     ClusterIP   10.109.30.23     <none>        5432/TCP   20h   app=orders-db
PS C:\Users\jyoti.s07\Desktop\QuickBite>
```

A11 Dashboard Deployments

E05_Kubernetes_Dashboard_Deployments.png

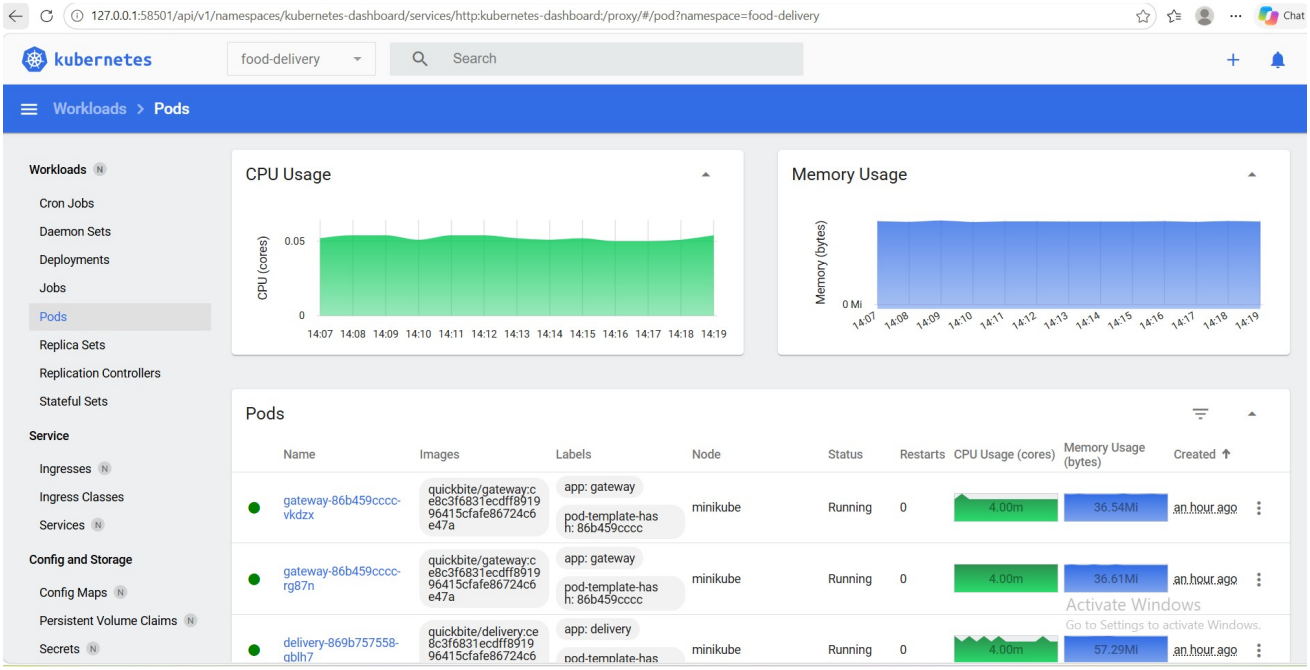
The Dashboard Deployments view shows the workloads in food-delivery and their readiness. A09 provides the matching CLI view.



A12 Dashboard Pods

E06_Kubernetes_Dashboard_Pods.png

The Dashboard Pods view exposes the workload instances and their current health. A09 provides the corresponding CLI inventory.



A14 Pod Disruption Budgets

E08_Pod_Disruption_Budget.png

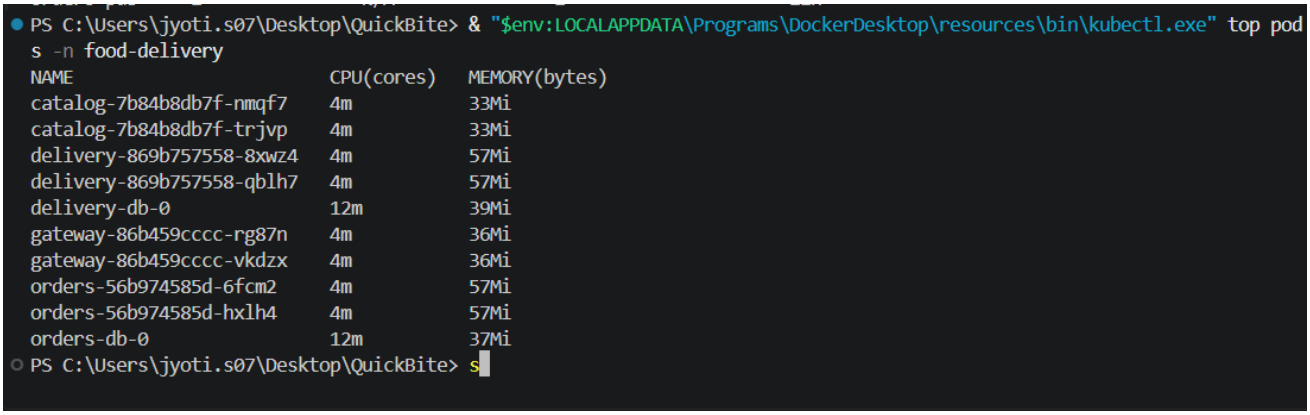
The PDB listing shows the voluntary disruption allowance. The captured configuration is **minAvailable 1** for the four application workloads.

```
PS C:\Users\jyoti.s07\Desktop\QuickBite> & "$env:LOCALAPPDATA\Programs\DockerDesktop\resources\bin\kubectl.exe" get pdb
  -n food-delivery
NAME           MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
catalog-pdb    1               N/A               1                     21h
delivery-pdb   1               N/A               1                     21h
gateway-pdb    1               N/A               1                     21h
orders-pdb     1               N/A               1                     21h
PS C:\Users\jyoti.s07\Desktop\QuickBite>
```

A15 Pod CPU and memory

E09_CPU_Memory_Metrics.png

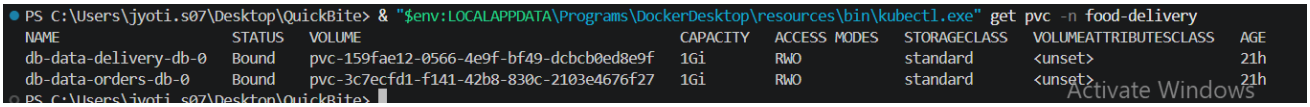
Resource metrics show CPU and memory per pod. These readings corroborate metrics availability for the HPA.



A16 PostgreSQL persistent claims

E10_PostgreSQL_PVC_Bound.png

The **two Bound PVCs** identify persistent storage for the Orders and Delivery PostgreSQL workloads. Each service retains its own database claim.

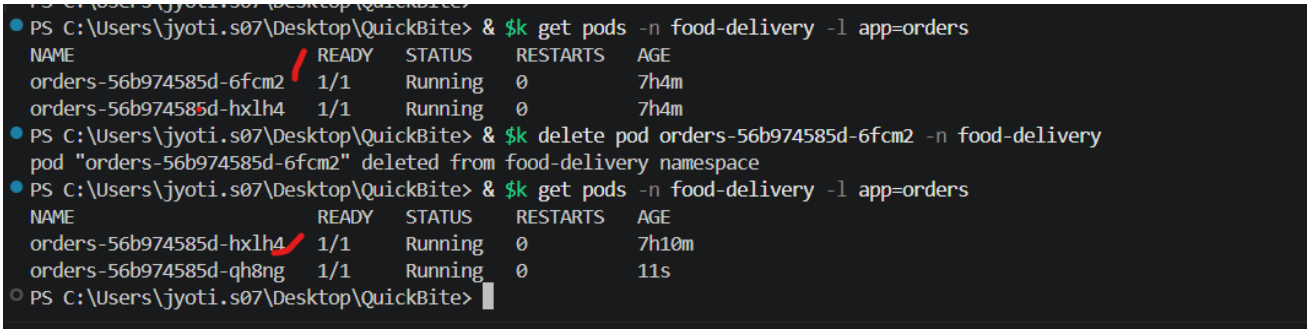


A14 records minAvailable 1 for the four application PDBs and one allowed voluntary disruption at the two-replica baseline. A40 shows an Orders pod being deleted, a replacement pod with a new name, and two running Orders pods afterward. The local Minikube topology demonstrates pod-level recovery.

A40 Orders pod deletion and replacement

06_Self_Healing_PDB.png

Recovery evidence records pod replacement and availability observations. Pod replacement comes from workload reconciliation.



7 Scalability enhancement using HPA

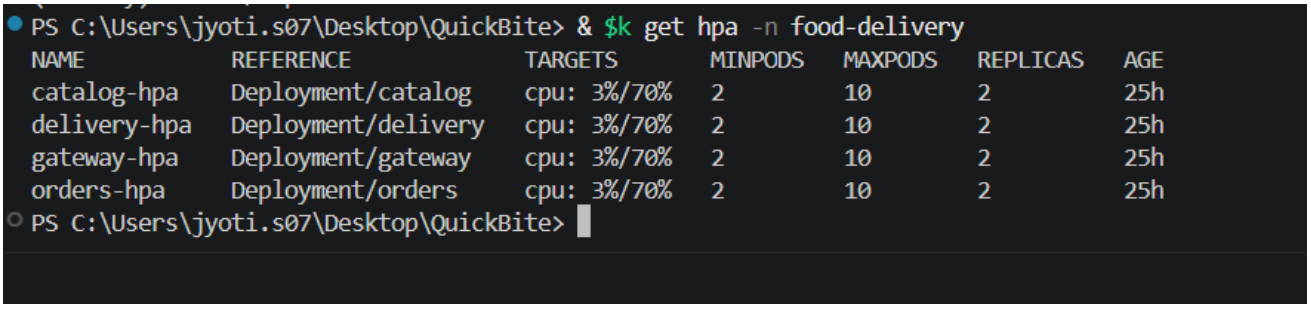
Observed scaling and recovery

A13 records all four application HPAs configured for 2–10 replicas with a 70% CPU target relative to CPU requests. A34 shows Orders at eight ready replicas under CPU stress, and A39 shows its return to two. B12 provides automatic-controller rescale events from a separate run.

A13 HPA policy baseline

E07_HPA_Autoscaling.png

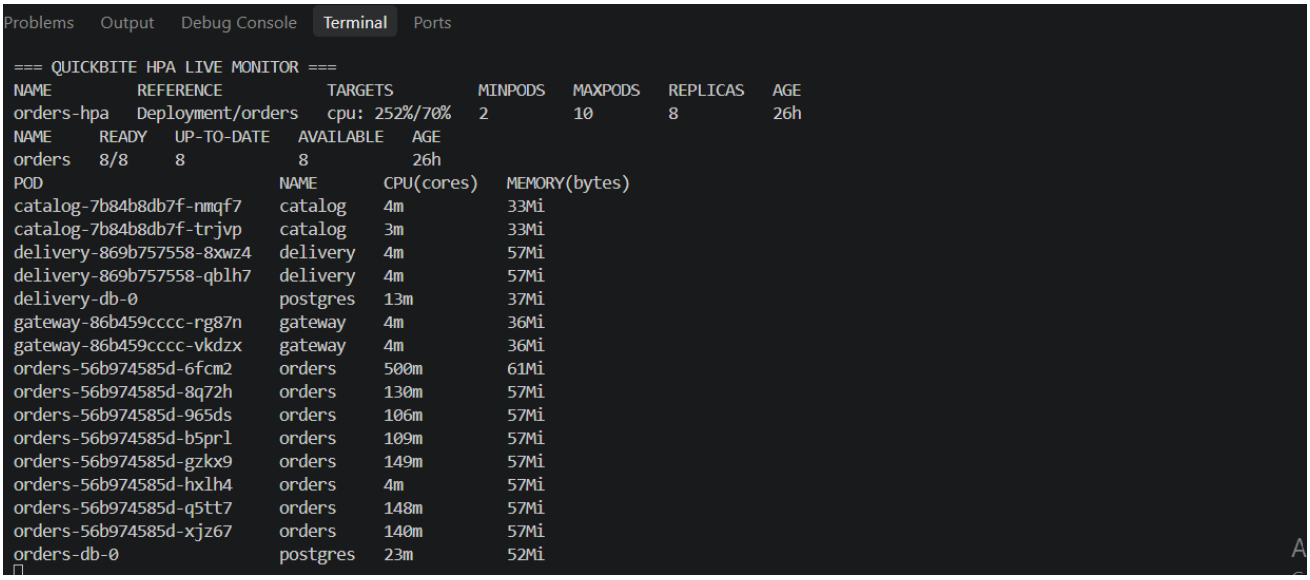
The HPA listing shows resource policies, CPU targets and replica bounds. A34 and A39 show observed scaling behavior.



A34 Orders scale out to eight replicas

E33A_HPA_Scale_Out_2_to_8.png

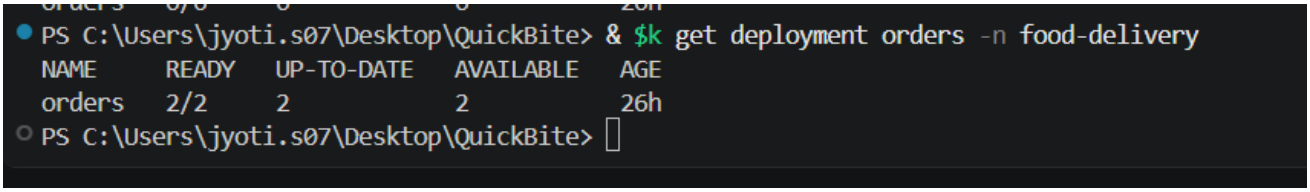
Orders reports **eight ready replicas**, with CPU utilization at **252%** against the 70% target. The HPA maximum is ten.



A39 Orders scale in to two replicas

E33B_HPA_Scale_In_8_to_2.png

After load subsides, the Orders Deployment returns from eight replicas to two in the recorded follow-up.



B12 HPA controller rescale history

E07_HPA_Details.txt

Selected SuccessfulRescale events from E07_HPA_Details.txt confirm CPU-driven controller decisions in a separate test run. The source log records initial metrics API errors followed by valid CPU readings and these successful scaling events.

Age at capture	Desired replicas	Controller reason
33 minutes	4	CPU utilization above target
30 minutes	6	CPU utilization above target
24 minutes	4	All metrics below target
23 minutes	3	All metrics below target
21 minutes	2	All metrics below target

API and scaling checks

```
minikube kubectl -- get hpa orders-hpa -n food-delivery -w
minikube kubectl -- get deployment orders -n food-delivery -w
```

Use separate terminals for the two watches. A34 and A39 record the CPU stress test with Orders growing to eight replicas and returning to two. The configured maximum is

ten. These deployment commands are checked against the source and manifests; they were not executed against the Windows cluster during this document revision.

8 Repositories

https://github.com/erjyotisrvtv-lab/QuickBites

https://github.com/erjyotisrvtv-lab/QuickBites/tree/main/services/catalog

https://github.com/erjyotisrvtv-lab/QuickBites/tree/main/services/orders

https://github.com/erjyotisrvtv-lab/QuickBites/tree/main/services/delivery

https://github.com/erjyotisrvtv-lab/QuickBites/tree/main/services/gateway

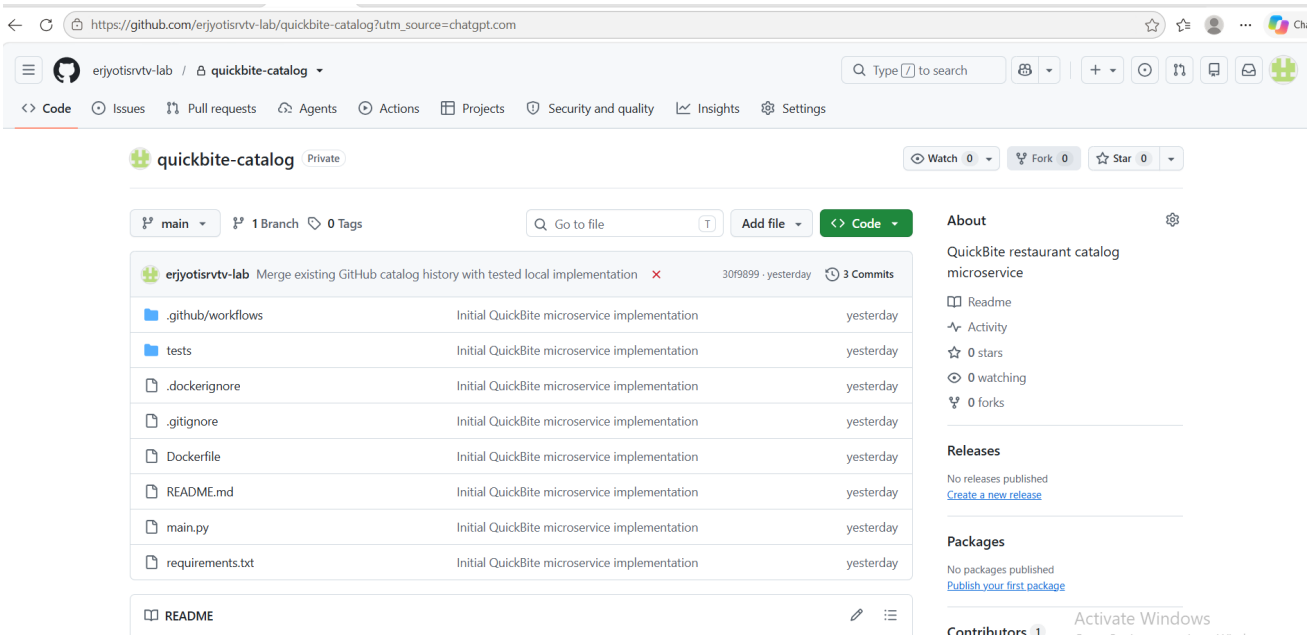
Use QuickBites as the single submission repository, with Catalog, Orders, Delivery and Gateway under services/. The original E28–E31 screenshots below document the earlier separate repositories and are retained as historical evidence. Current access links point to the consolidated service folders.

CI/CD verification GitHub Actions run 42574715176

A49 Historical independent Catalog repository

E28_Catalog_Independent_Repository.png.png

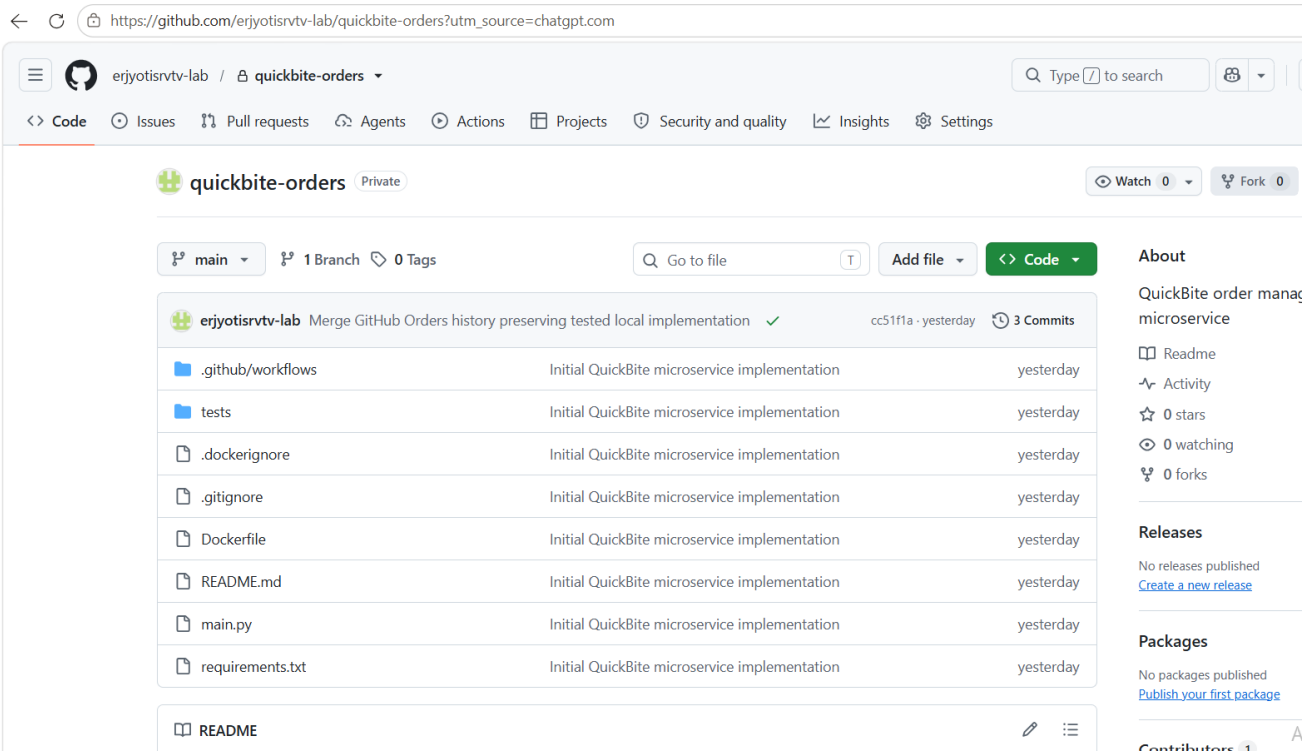
The Catalog repository screenshot documents separate source ownership for restaurant and menu APIs.



A50 Historical independent Orders repository

E29_Orders_Independent_Repository.png

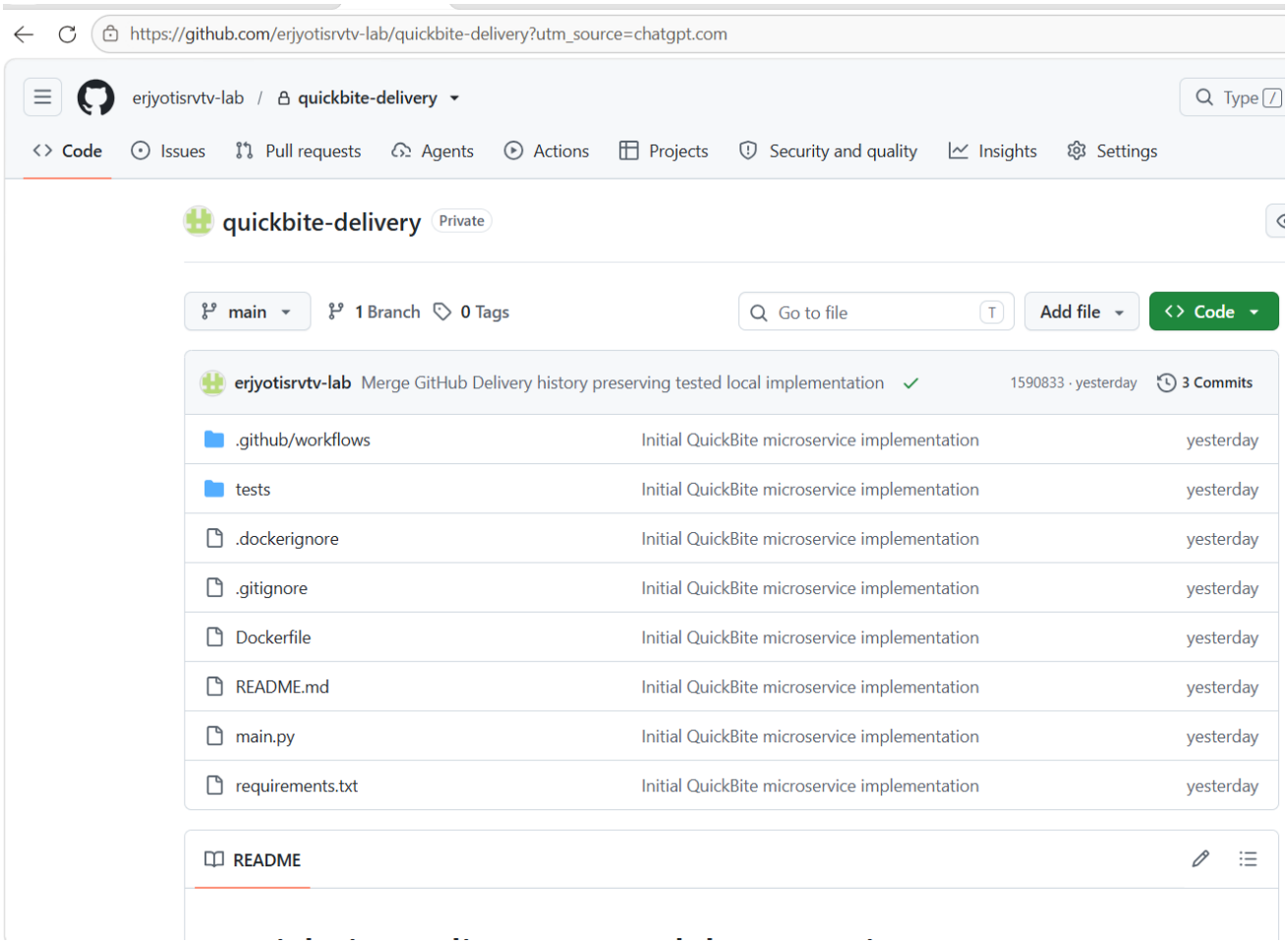
The Orders repository screenshot documents separate source ownership for order creation and lookup.



A51 Historical independent Delivery repository

E30_Delivery_Independent_Repository.png

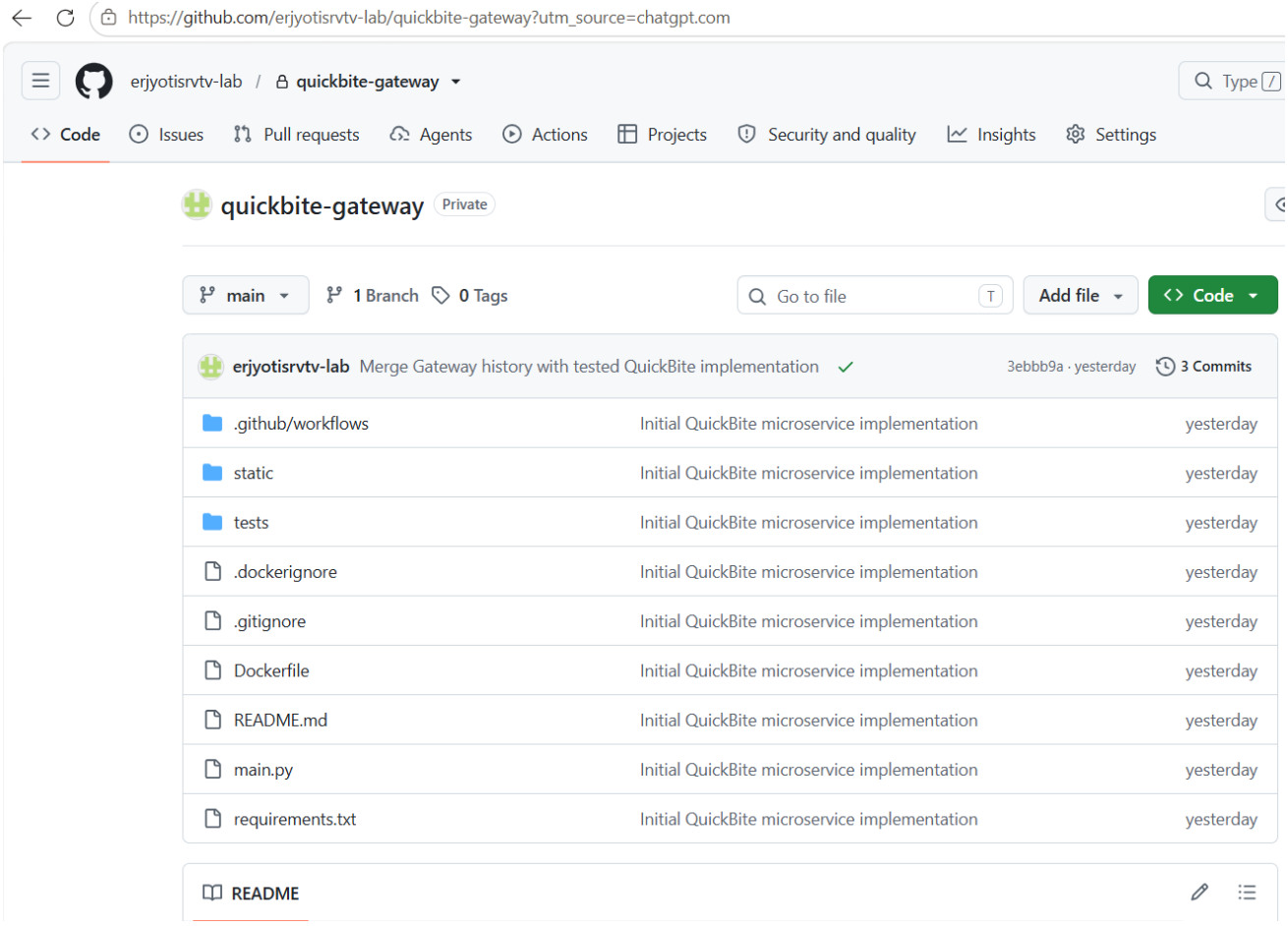
The Delivery repository screenshot documents separate source ownership for delivery operations.



A52 Historical independent Gateway repository

E31_Gateway_Independent_Repository.png

The Gateway repository screenshot documents separate source ownership for routing and the browser entry point.



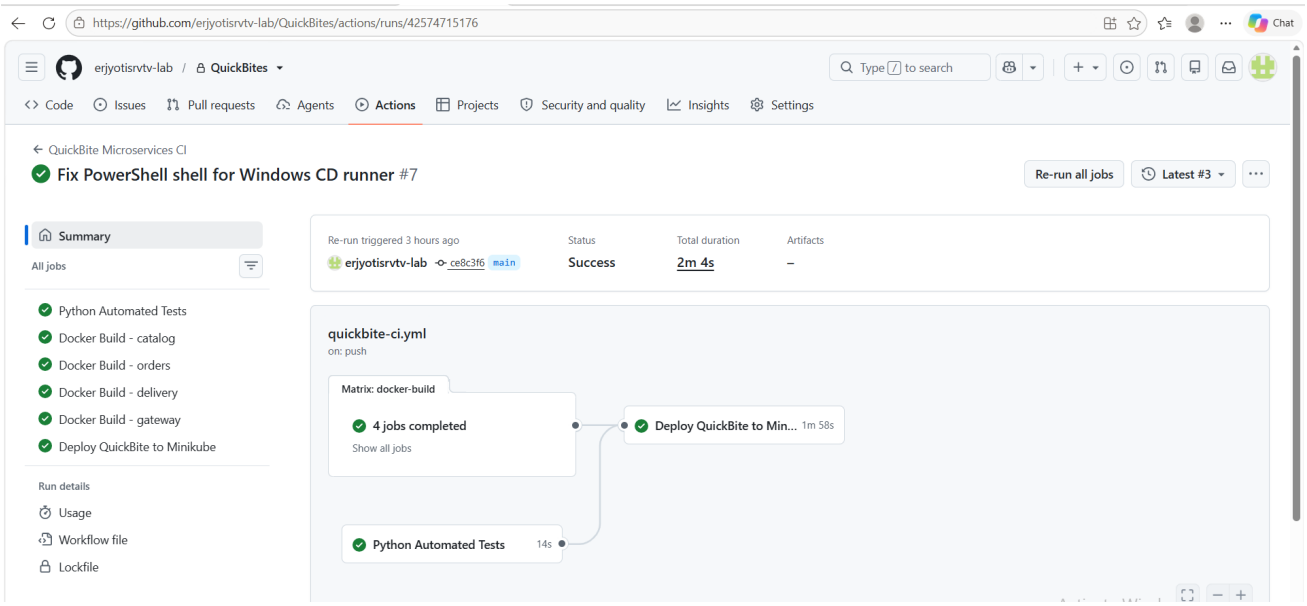
9 CI and CD execution

GitHub Actions Run 7 shows a successful Python test job, four Docker build jobs and a Kubernetes deployment job. The Windows self-hosted runner built or loaded deployment images into Minikube, updated the Deployment image references and waited for rollout. The four image references use the same commit tag, tying the deployment back to source revision ce8c3f6831ecdff891996415cfafe86724c6e47a. See E19–E22 and E32.

A43 Successful GitHub Actions run

E19_GitHub_Actions_CICD_Success.png

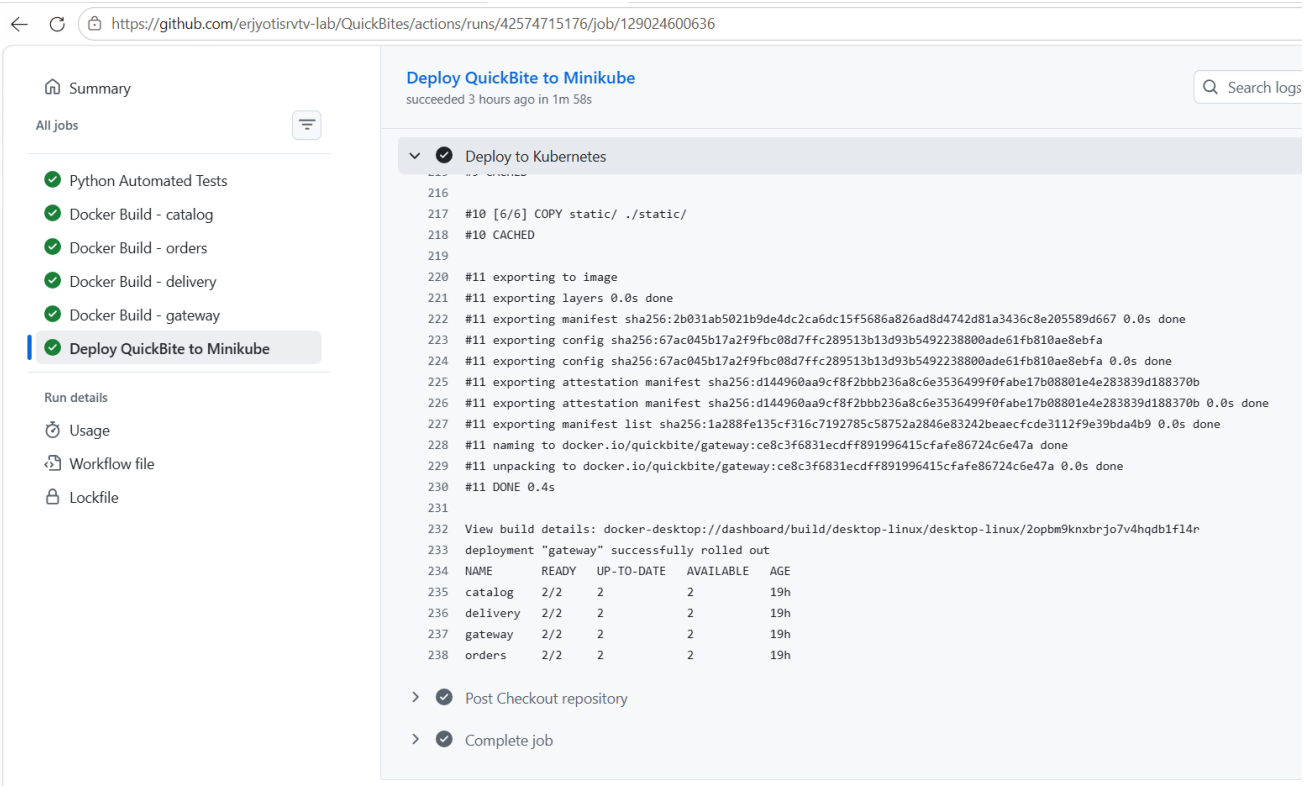
The Actions run overview shows the CI/CD job outcomes for the project.



A44 Kubernetes deployment job

E20_GitHub_Actions_Kubernetes_CD_Logs.png

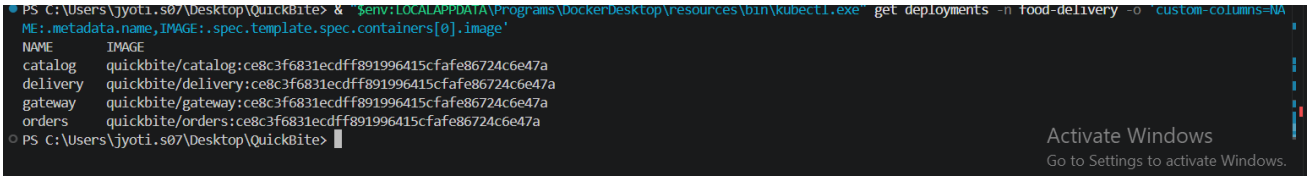
The deployment job log shows the Kubernetes delivery steps and rollout activity.



A45 Commit-tagged deployed images

E21_Deployed_Images_Git_Commit_SHA.png

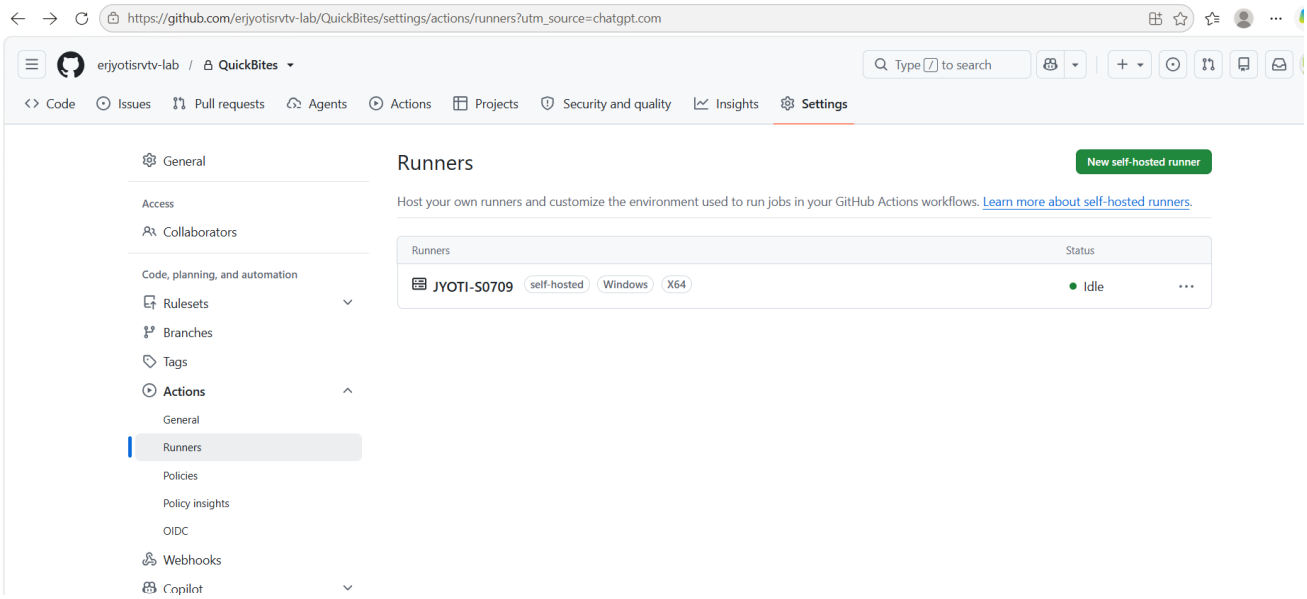
The Deployment image references carry commit-based tags, linking the runtime version to source history.



A46 Self-hosted runner

E22_GitHub_Self_Hosted_Runner.png

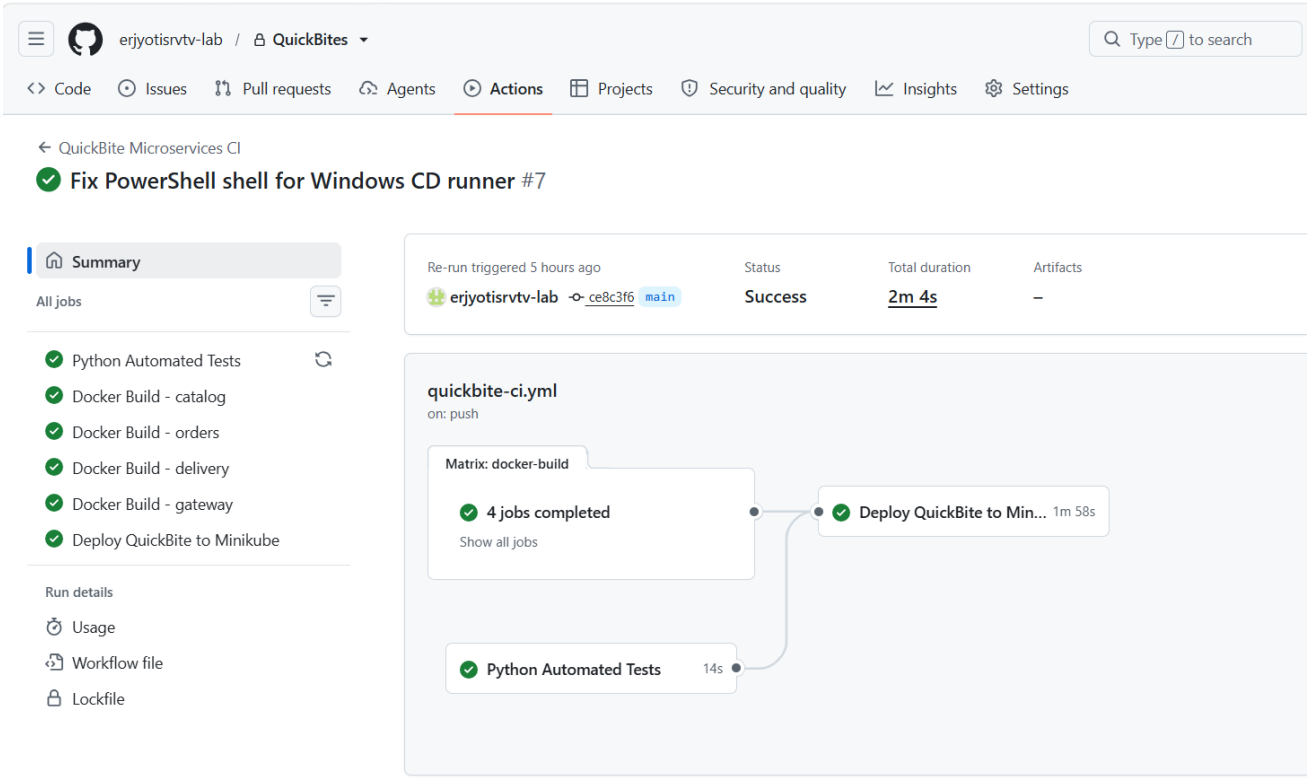
The GitHub runner view identifies the self-hosted Windows runner used for the local Minikube deployment.



A53 Python automated test results

E32_Python_Pytest_CI_Tests.png

The CI test capture shows pytest activity in the workflow. The Actions overview provides the run-level result.



References

Official technical documentation and project sources relevant to the implementation described in this report. Web references accessed 11 November 2026.

Technical documentation

[1] FastAPI. FastAPI documentation.
REST APIs and interactive API documentation (Sections 3–4).
https://fastapi.tiangolo.com/

[2] Docker. Docker Compose documentation.
Multi-container local execution and Compose lifecycle (Section 5).
https://docs.docker.com/compose/

[3] Kubernetes. minikube start.
Local Kubernetes installation and cluster setup (Section 6).
https://minikube.sigs.k8s.io/docs/start/

[4] Kubernetes. Horizontal Pod Autoscaling.
CPU utilization targets and replica scaling (Section 7).
https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

[5] Kubernetes. Specifying a Disruption Budget for your Application.
Pod Disruption Budget configuration (Section 6).
https://kubernetes.io/docs/tasks/run-application/configure-pdb/

[6] PostgreSQL Global Development Group. PostgreSQL 16 documentation.
Relational persistence for Orders and Delivery (Sections 2, 4 and 6).
https://www.postgresql.org/docs/16/

[7] GitHub. GitHub Actions documentation.
CI/CD workflows and runners (Section 9).
https://docs.github.com/en/actions

[8] Postman. Get started in Postman.
API requests, collections and response inspection (Section 3b).
https://learning.postman.com/docs/getting-started/overview/

Project sources and assignment brief

[9] Jyoti. QuickBites integration repository.
Application source and deployment artifacts (Sections 1–9).
https://github.com/erjyotisrvtv-lab/QuickBites

[10] Jyoti. QuickBites README.
Project setup and execution instructions (Sections 4–6).
https://github.com/erjyotisrvtv-lab/QuickBites/blob/main/README.md

[11] Jyoti. QuickBite CI/CD workflow.
Python tests, container builds and Kubernetes delivery (Section 9).
https://github.com/erjyotisrvtv-lab/QuickBites/blob/main/.github/workflows/quickbite-ci.yml

[12] Jyoti. GitHub Actions run 42574715176.
Project workflow run linked to the CI/CD evidence (Section 9).
https://github.com/erjyotisrvtv-lab/QuickBites/actions/runs/42574715176

[13] Jyoti. QuickBite service source directories.
Service folders within the QuickBites project repository (Section 8):
https://github.com/erjyotisrvtv-lab/QuickBites/tree/main/services/catalog
https://github.com/erjyotisrvtv-lab/QuickBites/tree/main/services/orders

https://github.com/erjyotisrvtv-lab/QuickBites/tree/main/services/delivery

https://github.com/erjyotisrvtv-lab/QuickBites/tree/main/services/gateway

[14] BITS Pilani WILP. SEZG583 Scalable Services. Microservices Development and Deployment assignment brief. Submission deadline: 12 November 2026.

Course handout supplied with the assignment: CCZG583_SS_Assignment(3).docx. Defines the 6/7/7 assessment structure and submission requirements.